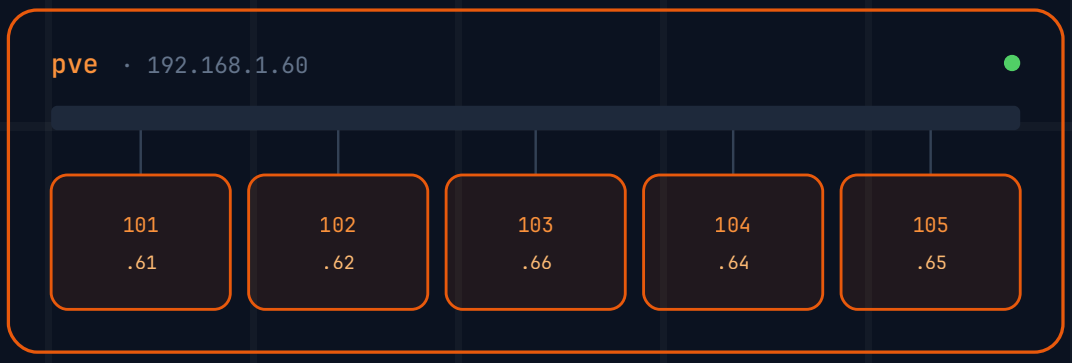
DOCUMENTACIÓN TÉCNICA

v1.6



Homelab casero

Mi servidor casero con Proxmox VE en un ASUS Chromebox 3, con un contenedor LXC para cada servicio

ESTA VERSIÓN INCLUYE

Proxmox VE 9.2 · Pi-hole · Vaultwarden · Tailscale · Uptime Kuma · Jellyfin

AUTOR
Alejandro

VERSIÓN
1.6

FECHA
04/11/2026

SERVIDOR
pve.home.arpa

Control del documento

Título	Homelab casero · Documentación técnica
Autor	Alejandro
Versión	1.6
Fecha	4 de noviembre de 2026
Estado	Lo sigo ampliando, cada vez que instalo un servicio nuevo le añado su capítulo
Qué incluye	La instalación de Proxmox VE de principio a fin, los servicios Pi-hole, Vaultwarden, Tailscale, Uptime Kuma y Jellyfin, y las mejoras que le hice a Pi-hole

CÓMO LEER ESTE DOCUMENTO

```
Dónde se ejecuta
# Aquí van los comandos
```

Arriba de cada bloque pone dónde hay que escribir el comando, por ejemplo en la Shell de Proxmox o en PowerShell en el PC

SALIDA REAL

```
Lo que me salió en el servidor
```

En los bloques claros está lo que me salió de verdad al poner los comandos, copiado tal cual

SEGURIDAD

En este documento no he puesto ninguna contraseña ni ningún correo, solo digo dónde los configuré, y como esta es la versión pública también he cambiado por valores de ejemplo el nombre de la tailnet, las IPs de Tailscale, los nombres de mis dispositivos, una dirección MAC y una huella SSH

Registro de cambios

VERSIÓN	FECHA	CAMBIOS
1.0	23/09/2026	La primera versión, con la explicación general del homelab, cómo preparé el USB, la instalación de Proxmox VE 9.2, lo que configuré después y las comprobaciones
1.1	23/09/2026	Añado el capítulo 6 con Pi-hole en el LXC 101, y también la copia de lo que tenía antes (apartado 2.1), el router y el DNS en el plan de red y palabras nuevas en el glosario
1.2	25/09/2026	Añado el capítulo 7 con Vaultwarden en el LXC 102 publicado con Tailscale, que desde aquí es lo que uso para entrar desde fuera (LXC 103), y actualizo el plan de red, el esquema y el glosario
1.3	25/09/2026	Añado el capítulo 8 con Tailscale como subnet router en el LXC 103 con la IP 192.168.1.66, y actualizo el plan de red, el esquema y el glosario
1.4	26/09/2026	Añado el capítulo 9 con Uptime Kuma en el LXC 104, con seis monitores y avisos por correo, y actualizo el plan de red, el esquema y el glosario
1.5	26/09/2026	Añado el capítulo 10 con Jellyfin en el LXC 105, que convierte el vídeo con la gráfica (Intel QuickSync) y guarda las películas en un volumen de 100 GiB del SSD, y actualizo el plan de red, el esquema y el glosario
1.6	04/11/2026	Amplío el capítulo 6 con las mejoras de Pi-hole (apartados 6.10 a 6.16), es decir más memoria, las listas HaGeZi Pro++, TIF y DoH, Unbound como DNS propio con DNSSEC, cosas para que nadie se salte Pi-hole y Pi-hole como DNS de la tailnet, y con todo esto en Blockmeter pasé del 49 % al 90 %

Índice

PARTE I · SERVIDOR BASE

1 **Visión general del homelab** 7

1.1 Objetivo 7

1.2 Hardware del servidor 7

1.3 Por qué lo monté así 8

1.4 Plan de red e IPs 9

2 **Preparación de la instalación** 10

2.1 Copia de la instalación anterior 10

2.2 Descarga de la ISO 10

2.3 Verificación de la descarga 11

2.4 Grabación en el pendrive 11

3 **Instalación de Proxmox VE** 12

3.1 Antes de empezar 12

3.2 Arranque desde el USB 12

3.3 Asistente de instalación 12

3.4 Por qué estos valores 13

3.5 Distribución del disco 14

4 **Configuración posterior** 16

4.1 Primer acceso a la interfaz web 16

4.2 Repositorios de paquetes 16

4.3 Actualización del sistema 17

5 **Verificación y estado final** 18

5.1 Versión instalada 18

5.2 Gráfica integrada 18

5.3 Resumen de comprobaciones 19

5.4 Ficha técnica del nodo 19

PARTE II · SERVICIOS

6 **Pi-hole** 20

6.1 Qué es y cómo funciona 20
6.2 Ficha del contenedor 21
6.3 Plantilla de Debian 13 21
6.4 Creación del contenedor 22
6.5 Arranque automático 23
6.6 Preparación del sistema 23
6.7 Instalación de Pi-hole 24
6.8 Verificación 25
6.9 Configuración del router 26
6.10 Cómo estaba antes de mejorarlo 28
6.11 Más memoria para el contenedor 29
6.12 Listas de bloqueo HaGeZi 29
6.13 Unbound, mi propio DNS 31
6.14 Que nadie se salte Pi-hole 35
6.15 Pi-hole también fuera de casa 35
6.16 Resultado final 37
6.17 Resumen 38

7 **Vaultwarden** 39

7.1 Qué es y cómo funciona 39
7.2 Ficha del contenedor 40
7.3 Creación del contenedor 41
7.4 Ajustes desde Proxmox 42
7.5 Preparación del sistema 42
7.6 Instalación de Docker 43
7.7 Tailscale y HTTPS 43
7.8 Instalación de Vaultwarden 44
7.9 Cuenta e importación 46
7.10 Aplicaciones de Bitwarden 46
7.11 Copia de seguridad provisional 47
7.12 Resumen 49

8 **Tailscale (acceso remoto)** 50

8.1 Qué es y cómo funciona 50
8.2 Ficha del contenedor 51
8.3 Creación del contenedor 52
8.4 Ajustes desde Proxmox 53
8.5 Preparación del sistema 53

8.6 Reenvío de tráfico 54
8.7 Instalación de Tailscale 54
8.8 Configuración en el panel 54
8.9 Verificación 55
8.10 Resumen 56

9 **Uptime Kuma** 57
9.1 Qué es y cómo funciona 57
9.2 Ficha del contenedor 58
9.3 Comprobación previa de la IP 58
9.4 Creación del contenedor 59
9.5 Ajustes desde Proxmox 60
9.6 Preparación del sistema 60
9.7 Instalación de Uptime Kuma 61
9.8 Servicio de arranque automático 61
9.9 Primer acceso 62
9.10 Tailscale en el contenedor 62
9.11 Monitores 63
9.12 Avisos por correo 63
9.13 Verificación 64
9.14 Resumen 65

10 **Jellyfin** 66
10.1 Qué es y cómo funciona 66
10.2 Ficha del contenedor 67
10.3 Comprobación previa de la IP 67
10.4 Creación del contenedor 68
10.5 Ajustes desde Proxmox 69
10.6 Preparación del sistema 69
10.7 Instalación de Jellyfin 70
10.8 Carpetas y subida de películas 70
10.9 Asistente inicial 71
10.10 Aceleración por hardware 71
10.11 Verificación 72
10.12 Resumen 73

ANEXOS

A **Glosario** 74

CAPÍTULO 01

Visión general del homelab

Qué es este homelab, con qué equipo funciona, por qué lo he montado así y cómo he repartido las IPs

1.1 Objetivo

Mi homelab es un servidor que tengo en casa con **Proxmox VE** instalado en un **ASUS Chromebox 3**, y la idea es que cada servicio vaya en su **propio contenedor LXC** con su propia IP, así todo queda separado y si un servicio falla o lo tengo que reinstalar los demás siguen funcionando sin problema

Los servicios que tengo ahora mismo son estos

- **Pi-hole**, que bloquea anuncios y rastreadores en toda la red de casa usando el DNS
- **Vaultwarden**, mi propio gestor de contraseñas, que funciona con las apps de Bitwarden
- **Tailscale**, para entrar a la red de casa desde fuera sin abrir puertos en el router
- **Uptime Kuma**, que vigila que todos los servicios estén funcionando
- **Jellyfin**, para ver películas y series, y que convierte el vídeo usando la gráfica

1.2 Hardware del servidor

Equipo	ASUS Chromebox 3
Procesador	Intel Celeron 3865U · 2 núcleos / 2 hilos · 1,8 GHz
Memoria RAM	16 GB
Almacenamiento	SSD de 480 GB (447,1 GiB)
Gráfica integrada	Intel HD Graphics 610, que tiene Quick Sync para el vídeo
Red	Por cable Ethernet (la interfaz `nic0`)
Firmware	Le quité el firmware original de ChromeOS y le puse uno UEFI para poder instalar cualquier sistema
Funcionamiento	Encendido todo el día, 24/7

1.3 Por qué lo monté así

Contenedores LXC y no máquinas virtuales

El procesador solo tiene 2 núcleos, así que es lo que más se queda corto en este equipo, y una **máquina virtual** es como un ordenador entero simulado con su propio sistema y su propio kernel, o sea que cada una gasta bastantes recursos, en cambio un **contenedor LXC** usa el mismo kernel que Proxmox, arranca en unos segundos y gasta muy poca RAM, y aun así cada contenedor tiene su propia IP, su propio Debian y sus propios programas

Un servicio en cada contenedor

Cada servicio va en su contenedor y así puedo reiniciar, actualizar, hacer copias o reinstalar uno sin tocar el resto, y además de un vistazo ya sé qué hay en cada IP

Docker solo donde hace falta

Vaultwarden oficialmente solo viene como imagen de Docker, así que en su contenedor sí que he puesto Docker, pero los demás servicios los he instalado directamente en su contenedor y sin Docker

Entrar desde fuera con Tailscale

Para entrar al homelab cuando no estoy en casa uso Tailscale, que es una VPN que funciona con WireGuard y no hace falta abrir ningún puerto en el router, y encima le da a Vaultwarden una dirección con HTTPS sin que yo me tenga que preocupar de los certificados

Todo a mano

Todo lo he instalado a mano y paso a paso, sin scripts que lo hagan todo solos, porque así entiendo lo que hace cada cosa y además me queda apuntado

Lo que dejo para más adelante

Los otros equipos que tengo (el Minisforum Z83-F y el ASUS Chromebox CN60) de momento están guardados y no forman parte del homelab, y TrueNAS lo dejo para más adelante porque el Chromebox 3 solo tiene sitio para un disco por dentro

1.4 Plan de red e IPs

Red local	`192.168.1.0/24`
Router / puerta de enlace	`192.168.1.1` · Movistar Router Smart WiFi 6 Go
Reservas DHCP	El router solo me deja reservar IPs a partir de la `192.168.1.33`
DNS de la red	`192.168.1.61` (Pi-hole), que lo reparte el router por DHCP, y los dispositivos que tienen Tailscale también lo usan estén donde estén (apartado 6.15)

ID	NOMBRE	TIPO	IP	SERVICIO	ESTADO
—	pve	Nodo	192.168.1.60	Proxmox VE	Instalado
101	pi-hole	LXC	192.168.1.61	Pi-hole	Instalado
102	vaultwarden	LXC	192.168.1.62	Vaultwarden	Instalado
103	tailscale	LXC	192.168.1.66	Tailscale	Instalado
104	uptime-kuma	LXC	192.168.1.64	Uptime Kuma	Instalado
105	jellyfin	LXC	192.168.1.65	Jellyfin	Instalado

CÓMO PONGO LOS NOMBRES

El último número del ID de cada contenedor es el mismo que el último número de su IP (101 → .61, 102 → .62...), así sé qué contenedor es cuál sin tener que mirarlo, y la única excepción es el 103 (`tailscale`), que usa la `192.168.1.66`

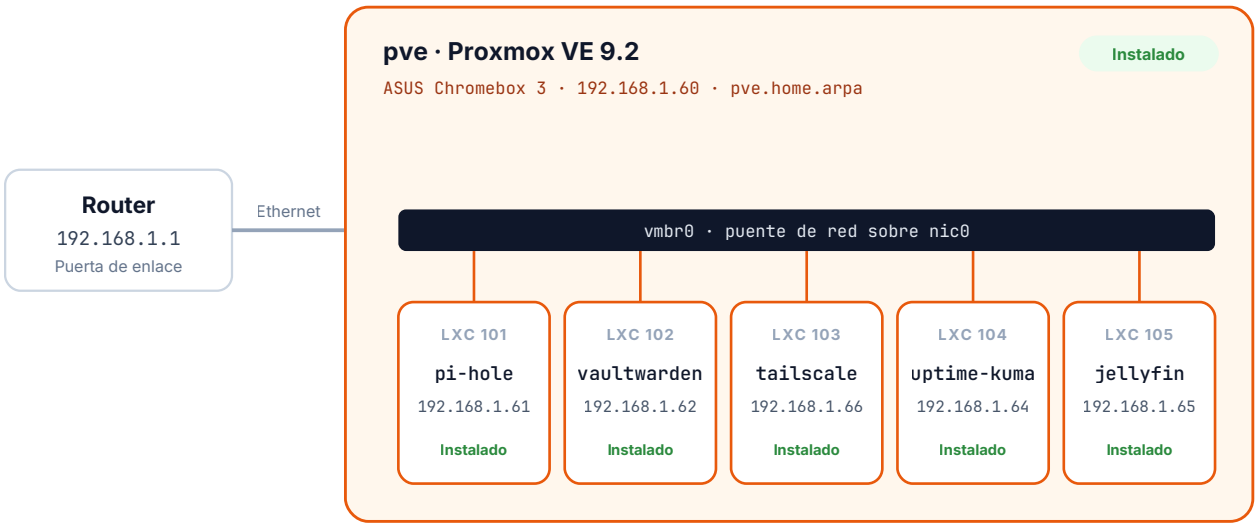


Figura 1 · Esquema de red del homelab, todos los contenedores se conectan a la red de casa por el puente `vmbr0`

CAPÍTULO 02

Preparación de la instalación

La copia de lo que tenía en el servidor antes de borrarlo, la descarga de Proxmox VE, cómo comprobé el archivo y cómo hice el pendrive de arranque

2.1 Copia de la instalación anterior

Antes de esto el servidor ya tenía un Proxmox con una máquina virtual Debian 13 donde funcionaban Pi-hole, Tailscale y Vaultwarden, y como al reinstalar se borra todo el disco, antes de empezar hice esto

QUÉ	QUÉ HICE	ESTADO
Contraseñas de Vaultwarden	Las exporté desde **Herramientas → Exportar caja fuerte** en formato **.json (Cifrado)** y tipo **«Protegido por contraseña»**, y luego las volví a importar al instalar Vaultwarden (capítulo 7)	Hecho
Configuración de Pi-hole	No la exporté porque preferí empezar Pi-hole de cero	Descartado
DNS del router	Lo dejé como estaba antes, porque si el router seguía dando el Pi-hole antiguo como DNS, al borrar el servidor toda la casa se habría quedado sin poder abrir webs	Hecho

POR QUÉ «PROTEGIDO POR CONTRASEÑA»

La otra opción es «Restringido a la cuenta», pero esa solo se puede importar en la misma cuenta de Vaultwarden y la cuenta desaparece al reinstalar, en cambio con «Protegido por contraseña» puedo importar el archivo en una cuenta nueva usando la contraseña que puse al exportar

2.2 Descarga de la ISO

La ISO la descargué de la web oficial de Proxmox (`proxmox.com/downloads`) y cogí la versión más nueva que había

Producto	Proxmox VE 9.2 ISO Installer	Versión	`9.2-1`
Archivo	`proxmox-ve_9.2-1.iso`	Tamaño	1,71 GB
Publicación	21 de mayo de 2026	Sistema base	Debian 13 «Trixie»

2.3 Verificación de la descarga

Antes de grabar la ISO miro su **hash SHA-256**, que es como la huella del archivo, y si la descarga se ha cortado o el archivo ha cambiado aunque sea un poco, el hash sale totalmente distinto, así que lo saco en el PC con PowerShell desde la carpeta donde está el archivo

PowerShell · PC con Windows

```
PS> Get-FileHash .\proxmox-ve_9.2-1.iso -Algorithm SHA256
```

Y el resultado tiene que ser exactamente igual que el hash que pone Proxmox en su web para la versión 9.2-1

```
4e88fe416df9b527624a175f24c9aa07c714d3332afb1ee3dbf3879573ef2c6c
```

2.4 Grabación en el pendrive

La ISO se graba en un pendrive de **4 GB** o más, pero ojo porque al grabarla **se borra todo lo que tenga el pendrive**, y para hacerlo hay dos programas

PROGRAMA	CÓMO SE USA
balenaEtcher (el que recomiendo)	Eliges la ISO, eliges el pendrive y le das a **Flash**
Rufus	Hay que elegir **«Modo imagen DD»**, porque con el otro modo puede que no arranque

CAPÍTULO 03

Instalación de Proxmox VE

La instalación desde cero en el SSD de 480 GB del Chromebox 3, con los valores exactos que puse en cada pantalla del instalador

3.1 Antes de empezar

Al Chromebox 3 le conecto estas cosas

- El **cable de red** directo al router, porque Proxmox necesita ir por cable ya que el WiFi no sirve como puente de red para los contenedores
- Un teclado y un monitor
- El pendrive con la instalación

3.2 Arranque desde el USB

1. Enciendo el equipo y le doy a **Esc** varias veces para que salga el menú de arranque
2. En el menú elijo el pendrive
3. En el menú de Proxmox elijo **Install Proxmox VE (Graphical)**

3.3 Asistente de instalación

El instalador tiene seis pantallas y en todas me salieron justo las opciones que esperaba

1. **Licencia (EULA)**, la acepto con **I agree**
2. **Target Harddisk**, elijo el SSD de 480 GB y en **Options** pongo el sistema de archivos **ext4**, y las opciones avanzadas del disco las dejo **por defecto** (lo explico en el apartado 3.5)
3. **Location and Time Zone**, aquí van el país, la zona horaria y el teclado
4. **Administration Password and Email**, la contraseña de `root` y el correo para los avisos del sistema
5. **Management Network Configuration**, la tarjeta de red, el nombre del equipo, la IP, la puerta de enlace y el DNS
6. **Summary**, reviso el resumen, dejo marcada la casilla para que se reinicie solo y le doy a **Install**, y cuando se reinicia quito el pendrive

Valores que puse

PANTALLA	CAMPO	VALOR
Target Harddisk	Disco	SSD de 480 GB
	Filesystem	`ext4`
	Opciones avanzadas	Por defecto
Location and Time Zone	Country	Spain
	Time zone	`Europe/Madrid`
	Keyboard Layout	Spanish
Password and Email	Contraseña de root	*Puesta, pero no la apunto aquí*
	Email	*Puesto, pero no lo apunto aquí*
Network Configuration	Management Interface	`nic0`
	Hostname (FQDN)	`pve.home.arpa`
	IP Address (CIDR)	`192.168.1.60/24`
	Gateway	`192.168.1.1`
	DNS Server	`1.1.1.1`

3.4 Por qué estos valores

EXT4 Y NO ZFS

ZFS está bien cuando tienes varios discos (por ejemplo para tener copias en espejo), pero gasta bastante más RAM y CPU, y con un solo disco no me aporta casi nada, así que ext4 es lo más ligero y lo más sencillo

EL DOMINIO HOME.ARPA

`home.arpa` es el dominio que está reservado de forma oficial para las redes de casa, y es mejor que inventarse uno como `.local`, ya que ese se usa para otra cosa y puede dar problemas con los nombres

DNS 1.1.1.1 Y NO PI-HOLE

Pi-hole va a ser un contenedor dentro de este mismo Proxmox, entonces si Proxmox usara Pi-hole como DNS, al arrancar se quedaría sin DNS hasta que el contenedor estuviera encendido, por eso el servidor usa un DNS de fuera (`1.1.1.1` , que es el de Cloudflare)

LA TARJETA DE RED SE LLAMA NIC0

Las versiones nuevas de Proxmox le ponen un nombre fijo a la tarjeta de red (`nic0`) para que no cambie después de actualizar el kernel, y encima de ella el instalador crea el puente `vmbr0`, que es el que tiene la IP del servidor y al que luego se conectan los contenedores

3.5 Distribución del disco

Con las opciones avanzadas por defecto el instalador divide el SSD en tres particiones, y la tercera la convierte en un grupo LVM que se llama `pve`, que a su vez se reparte en volúmenes para la swap, el sistema y los datos



Particiones

PARTICIÓN	TAMAÑO	PARA QUÉ ES	MONTAJE
1	1007 KiB	Arranque BIOS (por compatibilidad)	—
2	1 GiB	Partición EFI (para arrancar en UEFI)	/boot/efi
3	446,1 GiB	LVM, grupo `pve`	—

Volúmenes del grupo pve

VOLUMEN	TAMAÑO	PARA QUÉ ES	EN PROXMOX
swap	8 GiB	Memoria de reserva	—
root	96 GiB	El sistema, las plantillas, las ISOs y las copias locales, montado en `/`	local
data	319,6 GiB	Almacén *thin* para los discos de los contenedores y las VMs (más 3,3 GiB de metadatos)	local-lvm

Qué hace cada opción avanzada

OPCIÓN	QUÉ CONTROLA	VALOR POR DEFECTO	RESULTADO
`hdsize`	Cuánto disco usa Proxmox	Todo el disco	`446,1 GiB`
`swapsize`	El tamaño de la swap	Lo mismo que la RAM, pero como mínimo 4 GB y como máximo 8 GB	`8 GiB`
`maxroot`	El tamaño máximo del sistema	Como mucho la cuarta parte del disco	`96 GiB`
`minfree`	El espacio que deja libre en el grupo	16 GB si el disco tiene más de 128 GB	`≈16 GiB`
`maxvz`	El tamaño máximo del almacén de datos	Todo lo que sobra	`319,6 GiB`

¿POR QUÉ 447 Y NO 480?

Los fabricantes cuentan en GB (1 GB son 1000^3 bytes) y Linux cuenta en GiB (1 GiB son 1024^3 bytes), así que 480 GB son 447,1 GiB, o sea que es el mismo disco pero contado de otra forma

¿QUÉ QUIERE DECIR «THIN»?

En el almacén `local-lvm` un disco de 8 GiB de un contenedor no ocupa los 8 GiB de golpe, sino solo lo que tiene escrito de verdad, y así se aprovecha mucho mejor el espacio

CAPÍTULO 04

Configuración posterior

La primera vez que entro a la web de Proxmox, el cambio a los repositorios gratis y la actualización de todo el sistema

4.1 Primer acceso a la interfaz web

Proxmox se maneja desde el navegador de cualquier equipo de la red entrando en esta dirección

```
https://192.168.1.60:8006
```

La primera vez el navegador avisa de que el certificado no es de confianza, pero es normal porque Proxmox usa un certificado hecho por él mismo, así que le doy a **Avanzado → Continuar** y entro con estos datos

Usuario	`root`
Contraseña	La que puse en el instalador
Reino (Realm)	Linux PAM standard authentication
Idioma	Español

> **EL AVISO DE «NO HAY SUSCRIPCIÓN VÁLIDA»**
> Cuando inicias sesión sale este aviso y se cierra con **Aceptar**, y no quita ninguna función de Proxmox, solo te dice que no estás pagando el soporte oficial

4.2 Repositorios de paquetes

Proxmox viene con los repositorios **Enterprise** puestos, que solo funcionan si pagas una suscripción, y sin ella las actualizaciones fallan, así que para usarlo en casa hay que poner el repositorio gratis, que se llama **No-Subscription**

Esto está en **pve → Actualizaciones → Repositorios**

1. Selecciono el repositorio que apunta a `enterprise.proxmox.com/debian/pve` y le doy a **Deshabilitar**
2. Hago lo mismo con el repositorio **Enterprise de Ceph**
3. Le doy a **Añadir**, elijo **No-Subscription** y confirmo

REPOSITORIO	CÓMO QUEDA
Proxmox VE Enterprise	Deshabilitado
Ceph Enterprise	Deshabilitado
Proxmox VE No-Subscription	Habilitado
Repositorios de Debian 13 (sin tocar)	Habilitado

EL AVISO DEL NO-SUBSCRIPTION

La web te avisa de que este repositorio no se recomienda para empresas que lo usan en serio, pero para un homelab es lo normal

4.3 Actualización del sistema

Esto lo hago desde **pve → Shell**

```
Shell de Proxmox · root@pve
# apt update
# apt full-upgrade -y
```

`apt update`	Descarga la lista nueva de paquetes que hay en los repositorios
`apt full-upgrade -y`	Instala todas las actualizaciones, también los kernels nuevos y los cambios de dependencias, y el `-y` es para que diga que sí a todo solo

Y después de actualizar **tuve que reiniciar** el servidor

```
Shell de Proxmox · root@pve
# reboot
```

CAPÍTULO 05

Verificación y estado final

Las comprobaciones que hice después de instalarlo y la ficha del servidor tal y como queda

5.1 Versión instalada

Primero miro qué versión ha quedado instalada desde la Shell de Proxmox

Shell de Proxmox · root@pve

```
# pveversion
```

SALIDA REAL

```
pve-manager/9.2.20/49318c671b82f31e (running kernel: 7.0.14-19-pve)
```

pve-manager	9.2.20
Kernel en uso	7.0.14-19-pve

5.2 Gráfica integrada

Luego compruebo que el sistema ve la gráfica Intel, ya que más adelante la usa Jellyfin para convertir el vídeo

Shell de Proxmox · root@pve

```
# ls -l /dev/dri
```

SALIDA REAL

```
total 0
drwxr-xr-x 2 root root         80 Sep 23 12:08 by-path
crw-rw---- 1 root video  226,   1 Sep 23 12:08 card1
crw-rw---- 1 root render 226, 128 Sep 23 12:08 renderD128
```

renderD128	Es lo que se usa para acelerar el vídeo y lo que luego le paso al contenedor de Jellyfin, y es del grupo `render`
card1	Es la gráfica en sí, y que se llame `card1` en vez de `card0` es normal y no pasa nada

5.3 Resumen de comprobaciones

QUÉ HE COMPROBADO	RESULTADO
Entro a la web en `https://192.168.1.60:8006`	Correcto
Hay internet y DNS (la actualización bajó e instaló los paquetes)	Correcto
Proxmox VE 9.2 instalado y actualizado (`pve-manager 9.2.20`)	Correcto
El disco repartido con los valores por defecto en el SSD de 480 GB	Correcto
La gráfica Intel se ve bien (`/dev/dri/renderD128`)	Correcto

5.4 Ficha técnica del nodo

pve

`pve.home.arpa · 192.168.1.60`

Equipo	ASUS Chromebox 3 · Celeron 3865U · 16 GB RAM · SSD 480 GB
Sistema	Proxmox VE 9.2 (`pve-manager 9.2.20`) sobre Debian 13
Kernel	`7.0.14-19-pve`
Dirección IP	`192.168.1.60/24`
Puerta de enlace	`192.168.1.1`
DNS	`1.1.1.1`
Red	Tarjeta `nic0` · puente `vmbr0`
Sistema de archivos	ext4 sobre LVM
Almacenamiento	`local` · 96 GiB · `local-lvm` · 319,6 GiB (thin)
Repositorio	No-Subscription
Acceso web	`https://192.168.1.60:8006`

SIGUIENTE PASO

Capítulo 6 · Pi-hole · LXC 101 · 192.168.1.61

El primer servicio del homelab, que bloquea los anuncios en toda la red de casa

CAPÍTULO 06 · SERVICIO

Pi-hole

Bloqueo de anuncios, rastreadores y malware para toda la red de casa, metido en su propio contenedor LXC y mejorado después para que bloquee todo lo posible

6.1 Qué es y cómo funciona

Cada vez que un dispositivo abre una web o una app, primero le pregunta a un servidor **DNS** qué IP tiene cada dominio que necesita, y eso incluye también los de anuncios y rastreadores, entonces lo que hace Pi-hole es ponerse en medio para que todos los dispositivos de casa le pregunten a él

Si el dominio está en una **lista de bloqueo**, Pi-hole contesta `0.0.0.0`, que es una dirección que no lleva a ningún sitio, y el anuncio no llega a cargar, y si no está en la lista lo resuelve **Unbound**, que es un DNS que tengo en el mismo contenedor (apartado 6.13), y devuelve la respuesta normal

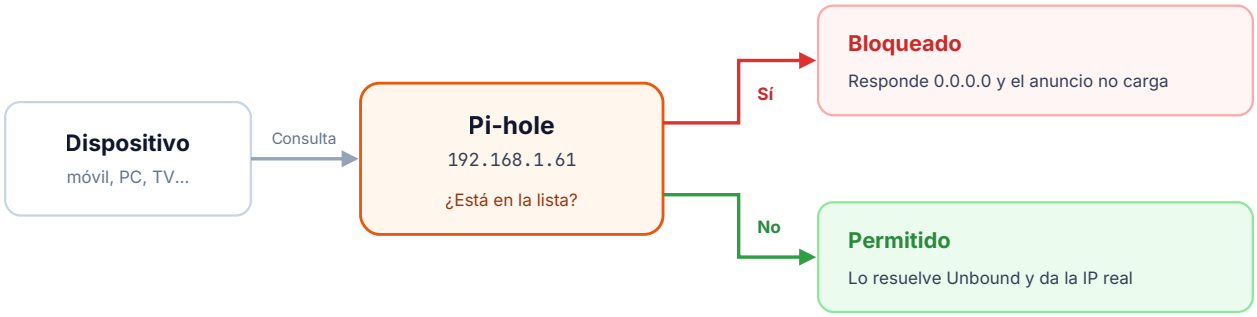


Figura 2 · Cómo decide Pi-hole si bloquea una consulta DNS o no

LO QUE NO PUEDE HACER

Pi-hole bloquea por el nombre del dominio, así que no puede quitar los anuncios que vienen del mismo servidor que el contenido, como los de YouTube o los del plan con anuncios de Netflix, y tampoco quita los avisos de cookies porque son parte de la propia web

6.2 Ficha del contenedor

ID	`101`
Nombre de host	`pi-hole`
Tipo	Contenedor LXC sin privilegios y con anidamiento
Sistema	Debian 13 (plantilla `debian-13-standard_13.1-2_amd64.tar.zst`)
Recursos	1 núcleo · 1024 MiB de RAM (al crearlo eran 512 MiB, luego lo amplié en el apartado 6.11) · 512 MiB de swap · disco de 8 GiB en `local-lvm`
Red	`eth0` en `vmbr0` · `192.168.1.61/24` · puerta de enlace `192.168.1.1`
Arranque	Se enciende solo al arrancar el servidor, y es el primero (orden 1)
Zona horaria	La misma que el servidor (`Europe/Madrid`)
Software	Pi-hole (Core v6.4.3 · Web v6.6 · FTL v6.7.1) · Unbound
Listas de bloqueo	StevenBlack · HaGeZi Pro++ · HaGeZi TIF · HaGeZi DoH
Resolución DNS	Unbound en el mismo contenedor (`127.0.0.1#5335`), comprobando DNSSEC
Clientes	Toda la red de casa (por el DHCP del router) y los dispositivos que tienen Tailscale
Panel web	`http://192.168.1.61/admin`

6.3 Plantilla de Debian 13

Los contenedores se crean a partir de una **plantilla**, que es como una imagen base del sistema ya preparada, y esta misma plantilla me sirve luego para el resto de contenedores del homelab

1. En la web de Proxmox voy a **pve → local (pve) → Plantillas CT → botón Plantillas**
2. Busco `debian-13-standard`, la selecciono y le doy a **Descargar**

Plantilla descargada	`debian-13-standard_13.1-2_amd64.tar.zst`

6.4 Creación del contenedor

Con el botón **Crear CT** (arriba a la derecha) se abre el asistente, y estos son los valores que puse en cada pestaña

PESTAÑA	CAMPO	VALOR
General	CT ID	`101`
	Nombre de host	`pi-hole`
	Contenedor sin privilegios	Sí
	Anidamiento	Sí
	Contraseña de root	*Puesta, pero no la apunto aquí*
Plantilla	Almacenamiento	`local`
	Plantilla	`debian-13-standard_13.1-2`
Discos	Almacenamiento	`local-lvm`
	Tamaño del disco	8 GiB
CPU	Núcleos	1
Memoria	Memoria	512 MiB (luego la subí a 1024 MiB)
	Swap	512 MiB
Red	Nombre / Puente	`eth0 / vmbr0`
	IPv4	Estático · `192.168.1.61/24`
	Puerta de enlace (IPv4)	`192.168.1.1`
	IPv6	Estático, vacío
DNS	Dominio y servidores	Vacíos (coge lo que tiene el servidor)
Confirmar	Iniciar después de creado	Sí

POR QUÉ «SIN PRIVILEGIOS»

En un contenedor sin privilegios el `root` de dentro no es el `root` de verdad del servidor, así que si alguien consiguiera entrar en el contenedor no podría controlar Proxmox, y además es la opción más segura y la que viene puesta por defecto

POR QUÉ UN DISCO DE 8 GIB

Pi-hole guarda un historial de consultas que va creciendo con el tiempo, por eso le dejo margen, y como `local-lvm` es de tipo *thin* el contenedor solo ocupa en el SSD lo que usa de verdad y no los 8 GiB enteros

6.5 Arranque automático

Esto se pone en **101 (pi-hole) → Opciones**

Iniciar al arrancar	Sí
Orden de inicio/apagado	1

POR QUÉ EL PRIMERO

Toda la red necesita Pi-hole para los nombres, así que cuando se enciende el servidor lo mejor es que el DNS sea lo primero que esté funcionando

6.6 Preparación del sistema

Desde **101 → Consola** y con el usuario `root` actualizo el sistema, instalo `curl` (que hace falta para bajar el instalador) y miro la hora

Consola del contenedor · root@pi-hole

```
# apt update && apt full-upgrade -y
# apt install -y curl
# date
```

Zona horaria

El contenedor no tenía la hora de España y en las opciones del contenedor de la web no hay nada para cambiarla, así que lo hice desde la **Shell de Proxmox** para que el contenedor use la misma zona horaria que el servidor

```
Shell de Proxmox · root@pve
# pct set 101 --timezone host
```

`pct`	Es la herramienta de Proxmox para manejar los contenedores desde la terminal
`set 101`	Cambia la configuración del contenedor 101
`--timezone host`	Hace que el contenedor use la zona horaria del servidor (`Europe/Madrid`)

> **OJO**
> Proxmox pone la zona horaria cuando arranca el contenedor, así que si `date` no la enseña al momento solo hay que reiniciarlo

6.7 Instalación de Pi-hole

Uso el instalador oficial, pero primero lo bajo a un archivo y luego lo ejecuto, porque así puedo mirar lo que hace antes de lanzarlo (por ejemplo con `less basic-install.sh`), y todo esto en la consola del contenedor

```
Consola del contenedor · root@pi-hole
# curl -sSL https://install.pi-hole.net -o basic-install.sh
# bash basic-install.sh
```

Lo que respondí en el instalador

PANTALLA	RESPUESTA	POR QUÉ
Bienvenida y donaciones	OK	Son solo pantallas de información
Static IP Needed	Continue	Es solo un aviso, la IP del contenedor ya es fija
Upstream DNS Provider	Cloudflare	Es el DNS de fuera al que Pi-hole manda lo que no bloquea, y más adelante lo cambié por Unbound (apartado 6.13)
Blocklists	Yes	Activa la lista de StevenBlack, que es una lista de bloqueo muy usada
Enable Logging	Yes	Guarda el registro de consultas para verlo en el panel
Privacy mode	0 · Show everything	Enseña todos los datos en el panel, total es la red de mi casa

Cuando termina, el instalador te da una contraseña aleatoria para el panel web, pero yo la cambié por una mía

```
●●● Consola del contenedor · root@pi-hole
# pihole setpassword
```

Y el panel de administración se abre en esta dirección

```
http://192.168.1.61/admin
```

6.8 Verificación

Versión instalada

Primero miro la versión desde la consola del contenedor

```
●●● Consola del contenedor · root@pi-hole
# pihole -v
```

SALIDA REAL

```
Core version is v6.4.3 (Latest: v6.4.3)
Web version is v6.6 (Latest: v6.6)
FTL version is v6.7.1 (Latest: v6.7.1)
```

Las tres partes de Pi-hole (el núcleo, la web y FTL, que es el que hace el DNS) están en la última versión

Resolución y bloqueo

Después desde el PC, con PowerShell, le pregunto directamente a Pi-hole por un dominio normal y por uno de publicidad

PowerShell · PC con Windows

```
PS> nslookup google.com 192.168.1.61
PS> nslookup doubleclick.net 192.168.1.61
```

SALIDA REAL · GOOGLE.COM

```
Servidor:  pi.hole
Address:  192.168.1.61

Respuesta no autoritativa:
Nombre:  google.com
Addresses:  2a00:1450:4003:816::200e
          216.58.205.46
```

SALIDA REAL · DOUBLECLICK.NET

```
Servidor:  pi.hole
Address:  192.168.1.61

Nombre:  doubleclick.net
Addresses:  ::
          0.0.0.0
```

DOMINIO	LO QUE CONTESTA PI-HOLE	RESULTADO
`google.com`	Sus direcciones de verdad, en IPv6 y en IPv4	Permitido
`doubleclick.net`	`::` y `0.0.0.0`, que son direcciones vacías en IPv6 y en IPv4	Bloqueado

6.9 Configuración del router

Para que todos los dispositivos de casa usen Pi-hole sin tener que configurarlos uno por uno, hago que el router reparta su IP como DNS por DHCP

Router	Movistar Router Smart WiFi 6 Go
Reserva DHCP	`192.168.1.61` para la MAC del contenedor 101 (la MAC se ve en **101 → Red**), y borré la reserva vieja que era de la máquina virtual de antes
DNS 1	`192.168.1.61`
DNS 2	Desactivado

Los dispositivos empiezan a usar Pi-hole cuando renuevan la conexión, en Windows se puede forzar con `ipconfig /release` y `ipconfig /renew` y en los móviles basta con quitar el WiFi y volver a ponerlo

POR QUÉ NO HAY UN SEGUNDO DNS

Si pusiera un segundo DNS de fuera (por ejemplo `1.1.1.1`), los dispositivos lo usarían a ratos y se colarían anuncios, en cambio con solo Pi-hole el bloqueo funciona siempre

SI EL SERVIDOR SE APAGA

Como solo hay un DNS, si el Chromebox se apaga o falla los dispositivos de casa no pueden abrir webs, y para tener internet mientras lo arreglo solo hay que poner el DNS del router en automático y luego volver a poner `192.168.1.61` cuando el servidor funcione

MEJORAS POSTERIORES

Optimización de Pi-hole

Cuando ya tenía todos los servicios del homelab funcionando, mejoré Pi-hole para que bloquee lo máximo posible sin romper las webs y apps que uso cada día, y en los apartados 6.10 a 6.16 explico cada cambio y lo que conseguí con él

6.10 Cómo estaba antes de mejorarlo

Para ver el antes y el después usé **Blockmeter** (`blockmeter.io`), que es una web que comprueba si tu dispositivo puede llegar a unos 380 dominios reales de anuncios, rastreadores, telemetría, estafas y malware, y la prueba la hice desde el navegador del PC sin ningún bloqueador instalado, así solo cuenta lo que bloquea Pi-hole

Resultado con la configuración del principio

Esto es lo que salió solo con la lista de StevenBlack, que es la que pone el instalador

CATEGORÍA	BLOQUEADOS	PORCENTAJE
Anuncios	47 de 54	87 %
Rastreadores y analítica	43 de 55	78 %
Telemetría y rastreadores de dispositivos	23 de 53	43 %
Estafas y malware	3 de 14	21 %
Otros dominios de listas	12 de 87	14 %
Contenido y filtros de salto (no cuenta para la nota)	1 de 117	–
Total · nota de Blockmeter	**129 de 380**	**49 %**

La nota solo tiene en cuenta las cinco primeras categorías, y la de contenido (VPN, redes sociales, apuestas, acortadores…) solo importa si quieres bloquear esas cosas a propósito

Lo que pensé mejorar

CAMBIO	PARA QUÉ	APARTADO
Más memoria para el contenedor	Las listas nuevas son mucho más grandes	6.11
Listas HaGeZi Pro++, TIF y DoH	Bloquear más anuncios, rastreadores, telemetría y malware, y que nadie se salte Pi-hole con DNS cifrado	6.12
Unbound	Resolver los nombres sin depender de Cloudflare y comprobar las respuestas con DNSSEC	6.13
Que nadie se salte Pi-hole	Que Firefox y el iPhone no se salten Pi-hole	6.14
Pi-hole como DNS de la tailnet	Que también bloquee fuera de casa con Tailscale	6.15

LO QUE NO QUISE PONER

- **La lista Ultimate de HaGeZi**, que es la más agresiva, y su propio autor dice que tiene mucho riesgo de romper cosas, porque limita Facebook y Messenger y puede hacer que las webs detecten mal tu país, que salgan más CAPTCHAs o que algunas funciones no vayan
- **La lista Bypass Full**, que bloquea las VPN y los proxys, pero también bloquearía Tailscale y me quedaría sin poder entrar desde fuera, así que solo bloqueo los servidores de DNS cifrado (la lista DoH)

6.11 Más memoria para el contenedor

Las listas nuevas tienen muchísimos más dominios que la de StevenBlack y Pi-hole los carga en memoria al actualizarlas, entonces con 512 MiB podía ir justo y lo subí a 1 GiB desde la Shell de Proxmox

Shell de Proxmox · root@pve

```
# pct set 101 --memory 1024
```

El cambio se aplica al momento y no hace falta reiniciar el contenedor

6.12 Listas de bloqueo HaGeZi

Usé las listas de **HaGeZi**, que es un proyecto muy conocido que tiene sus listas en cinco niveles, desde *Light* hasta *Ultimate*, y para alguien que puede desbloquear algo cuando haga falta su autor recomienda juntar **Pro++** con **TIF**, y como Pro++ ya bloquea casi todos los rastreadores de las marcas (teles, móviles, Windows...) no hace falta poner sus listas *Native* aparte

En el panel de Pi-hole, en **Lists**, las fui añadiendo una por una poniendo la dirección en **Address**, el nombre en **Comment** y dándole a **Add blocklist**

COMENTARIO	DIRECCIÓN	QUÉ BLOQUEA
StevenBlack	*La que pone el instalador*	Anuncios y malware, y la dejo puesta
HaGeZi Pro++	`https://raw.githubusercontent.com/hagezi/dns-blocklists/main/adblock/pro.plus.txt`	Anuncios, rastreadores, telemetría y phishing
HaGeZi TIF	`https://raw.githubusercontent.com/hagezi/dns-blocklists/main/adblock/tif.txt`	Malware, estafas y phishing, y se actualiza muy a menudo
HaGeZi DoH	`https://raw.githubusercontent.com/hagezi/dns-blocklists/main/adblock/doh.txt`	Servidores de DNS cifrado, para que nadie se salte Pi-hole

Actualización de Gravity

Gravity es lo que usa Pi-hole para bajar todas las listas y juntarlas en su base de datos, y yo lo lancé desde **Tools → Update Gravity → Update**, aunque también se puede hacer desde la consola del contenedor

Consola del contenedor · root@pi-hole

```
# pihole -g
```

Cuando acaba, en el panel principal sale en **Domains on Lists** el total de dominios de todas las listas, y los que se repiten entre listas no cuentan doble, y además Gravity se vuelve a ejecutar solo cada semana para tener las listas al día

POR QUÉ EL FORMATO ADBLOCK

HaGeZi tiene cada lista en varios formatos, y para Pi-hole uso el formato *Adblock* (`||dominio^`) porque cada línea bloquea el dominio y también todos sus subdominios

SI ALGO DEJA DE FUNCIONAR

Pro++ es más estricta que Pro y alguna vez puede bloquear el enlace de una oferta, el seguimiento de un correo de publicidad o alguna función de una app, pero se arregla en un minuto buscando en **Query Log** el dominio bloqueado (sale en rojo) y dándole a **Allow**

6.13 Unbound, mi propio DNS

Hasta ahora Pi-hole le pasaba a Cloudflare todas las consultas que no bloqueaba, o sea que Cloudflare veía todas las webs que se visitan en mi casa, y con **Unbound** Pi-hole las resuelve él solo, ya que pregunta directamente a los **servidores raíz** de internet, después a los del dominio de primer nivel (`.net`, `.com`...) y al final al servidor del propio dominio, y además comprueba con **DNSSEC** que nadie ha cambiado las respuestas por el camino

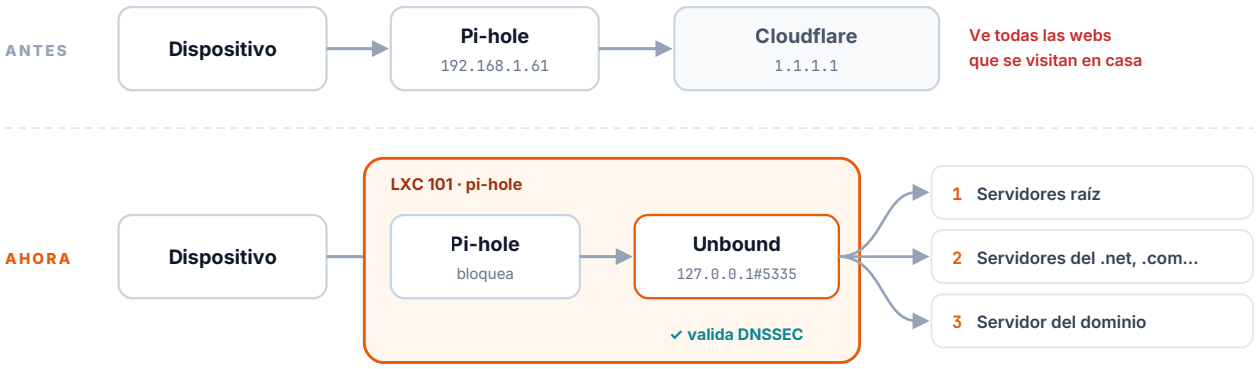


Figura 3 · Antes Pi-hole le pasaba a Cloudflare lo que no bloqueaba, y ahora lo resuelve Unbound él solo preguntando directamente a los servidores de internet

Instalación y configuración

En la consola del contenedor instalo Unbound y creo su archivo de configuración

Consola del contenedor · root@pi-hole

```
# apt install -y unbound dnsutils
# nano /etc/unbound/unbound.conf.d/pi-hole.conf
```

Y dentro del archivo pongo esto

/etc/unbound/unbound.conf.d/pi-hole.conf

```
server:
    verbosity: 0
    interface: 127.0.0.1
    port: 5335
    do-ip4: yes
    do-udp: yes
    do-tcp: yes
    do-ip6: no
    prefer-ip6: no
    harden-glue: yes
    harden-dnssec-stripped: yes
    use-caps-for-id: no
    edns-buffer-size: 1232
    prefetch: yes
    num-threads: 1
    private-address: 192.168.0.0/16
    private-address: 169.254.0.0/16
    private-address: 172.16.0.0/12
    private-address: 10.0.0.0/8
    private-address: fd00::/8
    private-address: fe80::/10
```

`interface / port`	Unbound solo escucha dentro del contenedor (`127.0.0.1`) y en el puerto `5335`, porque el 53 ya lo usa Pi-hole
`do-ip6: no`	La red de casa va por IPv4
`harden-glue / harden-dnssec-stripped`	Rechaza las respuestas raras y las que llegan sin la firma DNSSEC que tendrían que llevar
`edns-buffer-size: 1232`	Es el tamaño de paquete que se recomienda para que no haya problemas al partir las respuestas grandes
`prefetch: yes`	Renueva los dominios que más se usan antes de que caduquen, para que sigan saliendo rápido de la caché
`private-address`	No deja que un dominio de internet conteste con IPs de una red privada (es una protección contra el *DNS rebinding*)

Es la configuración de la guía oficial de Pi-hole para Unbound pero sin la opción `so-rcvbuf`, porque dentro de un contenedor solo da un aviso

Consola del contenedor · root@pi-hole

```
# systemctl restart unbound
# systemctl is-active unbound-resolvconf.service
# Esto solo si el comando de antes dice "active"
# systemctl disable --now unbound-resolvconf.service
```

En Debian el servicio `unbound-resolvconf` puede cambiar la configuración DNS del propio contenedor y liarla con Pi-hole, así que miro si está activo y si lo está lo desactivo

Verificación de Unbound

Le hago tres consultas directamente a Unbound, una a un dominio normal, otra a uno que tiene la firma DNSSEC rota a propósito y otra a uno con la firma bien

Consola del contenedor · root@pi-hole

```
# dig pi-hole.net @127.0.0.1 -p 5335
# dig fail01.dnssec.works @127.0.0.1 -p 5335
# dig dnssec.works @127.0.0.1 -p 5335
```

SALIDA REAL (FRAGMENTO) · PI-HOLE.NET

```
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 42928
;; flags: qr rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1
;; ANSWER SECTION:
pi-hole.net.            3600    IN      A       162.244.93.14
;; Query time: 531 msec
```

SALIDA REAL (FRAGMENTO) · FAIL01.DNSSEC.WORKS

```
;; ->>HEADER<<- opcode: QUERY, status: SERVFAIL, id: 52628
;; flags: qr rd ra; QUERY: 1, ANSWER: 0, AUTHORITY: 0, ADDITIONAL: 1
;; Query time: 1413 msec
```

SALIDA REAL (FRAGMENTO) · DNSSEC.WORKS

```
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 33785
;; flags: qr rd ra ad; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1
;; ANSWER SECTION:
dnssec.works.           3600    IN      A       46.23.92.212
;; Query time: 39 msec
```

DOMINIO	RESULTADO	QUÉ QUIERE DECIR
pi-hole.net	`NOERROR` y una IP	Unbound resuelve los nombres él solo, sin Cloudflare
fail01.dnssec.works	`SERVFAIL`	Rechaza la respuesta porque la firma DNSSEC está rota
dnssec.works	`NOERROR` con la marca `ad`	Ha comprobado que la firma DNSSEC está bien

POR QUÉ LA PRIMERA CONSULTA TARDÓ 531 MS

Unbound empezaba sin nada guardado y tuvo que hacer todo el camino desde los servidores raíz, pero las siguientes consultas al mismo dominio salen de su caché y tardan casi 0 ms

Pi-hole usa Unbound

En el panel de Pi-hole, en **Settings → DNS**, hago esto

1. Quito todas las casillas de **Cloudflare**
2. En **Custom DNS servers** escribo `127.0.0.1#5335`
3. Le doy a **Save**

Y para comprobar que todo el camino (dispositivo → Pi-hole → Unbound) comprueba DNSSEC, lo pruebo desde el PC con PowerShell

PowerShell · PC con Windows

```
PS> nslookup dnssec.works
PS> nslookup fail01.dnssec.works
```

El primero devuelve su IP y el segundo falla, igual que cuando le preguntaba directamente a Unbound

6.14 Que nadie se salte Pi-hole

Algunos programas pueden usar su propio DNS cifrado y saltarse Pi-hole, pero Pi-hole 6 ya trae unas protecciones para evitarlo, y yo comprobé que estaban activadas en **Settings → All settings → DNS**, en el apartado `dns.specialDomains`

OPCIÓN	ESTADO	QUÉ HACE
`mozillaCanary`	Activada	Le dice a Firefox que no use su DNS cifrado en esta red
`iCloudPrivateRelay`	Activada	Desactiva la Retransmisión privada de iCloud en esta red, que en Safari se saltaría Pi-hole
`designatedResolver`	Activada	Hace que los dispositivos no se cambien por su cuenta a un DNS cifrado

Y a esto se le suma la lista **HaGeZi DoH** del apartado 6.12, que bloquea los servidores de DNS cifrado más conocidos

> **LO QUE NO SE PUEDE EVITAR**
>
> Algunos aparatos, como algunos Chromecast o teles, llevan un DNS puesto por la marca y esos se pueden seguir saltando Pi-hole, y para evitarlo haría falta un router que pueda redirigir todo el tráfico DNS

6.15 Pi-hole también fuera de casa

Cuando estaba fuera de casa con Tailscale activado, el móvil seguía usando el DNS de la red donde estuviera (por ejemplo el de la compañía de datos) y me salían anuncios, así que ahora Tailscale le da Pi-hole como DNS a todos los dispositivos de la tailnet, y las consultas llegan a Pi-hole pasando por el subnet router (LXC 103, capítulo 8)

Esto se hace en el panel de Tailscale (`login.tailscale.com`), en la parte de **DNS**

1. En **Nameservers** le doy a **Add nameserver → Custom** y pongo la IP `192.168.1.61`
2. En esa misma ventana dejo desactivados **Restrict to domain** y **Use with exit node** y le doy a **Save**
3. Activo **Override DNS servers**
4. **MagicDNS** y **HTTPS Certificates** los dejo activados porque los necesita Vaultwarden

Nameserver	`192.168.1.61`
Restrict to domain (Split DNS)	No, porque quiero que Pi-hole se use para todos los dominios y no solo para algunos
Use with exit node	No, porque en mi homelab no uso *exit nodes*
Override DNS servers	Sí, así los dispositivos pasan del DNS de la red donde estén y usan Pi-hole
MagicDNS · HTTPS Certificates	Sí, sin tocar nada

Prueba con datos móviles

Con el iPhone con datos móviles y Tailscale activado, `http://doubleclick.net` ya no carga, y en el **Query Log** de Pi-hole esa consulta sale como hecha por `192.168.1.66`, que es el subnet router, porque es por donde llegan a casa las consultas de los dispositivos que están fuera

EL MÓVIL SE ACORDABA DE LA DIRECCIÓN

La primera vez la página sí cargó porque el iPhone se acordaba de la dirección de otra vez que entré y no llegó a preguntarle a Pi-hole, pero poniendo el modo avión unos segundos o usando una pestaña privada se borra eso

SI EL SERVIDOR SE APAGA

Ahora los dispositivos con Tailscale activado también dependen de Pi-hole estén en casa o fuera, así que si el Chromebox está apagado se quedan sin poder abrir webs, y lo que hay que hacer es quitar Tailscale en ese dispositivo hasta que el servidor vuelva

6.16 Resultado final

Volví a pasar Blockmeter desde el PC igual que la primera vez, y además Blockmeter también detecta qué nivel de las listas de HaGeZi estás usando y pasó de «ninguno» a **PRO++**

CATEGORÍA	ANTES	AHORA
Anuncios	47 de 54 · 87 %	**54 de 54 · 100 %**
Rastreadores y analítica	43 de 55 · 78 %	**55 de 55 · 100 %**
Telemetría y rastreadores de dispositivos	23 de 53 · 43 %	**45 de 53 · 85 %**
Estafas y malware	3 de 14 · 21 %	**14 de 14 · 100 %**
Otros dominios de listas	12 de 87 · 14 %	**65 de 87 · 75 %**
Contenido y filtros de salto (no cuenta para la nota)	1 de 117	8 de 117
Total	**129 de 380**	**241 de 380**
Nota de Blockmeter	**49 %**	**90 % (+41 puntos)**

Los 8 dominios bloqueados en la categoría de contenido son los servidores de DNS cifrado de la lista DoH, que es justo lo que quería bloquear de esa categoría

QUÉ ME PARECE EL RESULTADO

Anuncios, rastreadores y estafas y malware quedan al 100 %, y lo que todavía pasa (6 dominios de telemetría y 21 de «otros») solo lo bloquearía la lista Ultimate pero rompiendo apps y webs normales, así que me quedo con Pro++, y la categoría de contenido la dejo así a propósito porque bloquear más sería bloquear VPN (también Tailscale), redes sociales o apuestas

6.17 Resumen

QUÉ HE COMPROBADO	RESULTADO
El contenedor 101 creado con los valores que tenía pensados (nombre de host `pi-hole`)	Correcto
Arranque automático puesto, el primero (orden 1)	Configurado
La zona horaria del contenedor es la misma que la del servidor	Correcto
Pi-hole en la última versión (Core v6.4.3 · Web v6.6 · FTL v6.7.1)	Correcto
Resuelve los dominios normales y bloquea los de publicidad	Correcto
Reserva DHCP de la `.61` y el DNS de la red apuntando a Pi-hole en el router	Configurado
Memoria del contenedor subida a 1024 MiB	Configurado
Listas HaGeZi Pro++, TIF y DoH añadidas y Gravity actualizado	Correcto
Unbound funcionando como mi propio DNS y comprobando DNSSEC	Correcto
Las protecciones para que nadie se salte Pi-hole están activadas	Correcto
Pi-hole como DNS de la tailnet, así que también bloquea fuera de casa	Correcto
Blockmeter pasa del 49 % al 90 %	Correcto

SIGUIENTE PASO

Capítulo 7 · Vaultwarden · LXC 102 · 192.168.1.62

Mi propio gestor de contraseñas, recuperando la copia que exporté antes de reinstalar

CAPÍTULO 07 · SERVICIO

Vaultwarden

Mi propio gestor de contraseñas, que funciona con las apps de Bitwarden y está publicado con HTTPS usando Tailscale

7.1 Qué es y cómo funciona

Vaultwarden es un servidor que funciona igual que Bitwarden pero mucho más ligero que el oficial, y las contraseñas se cifran en el propio dispositivo con la **contraseña maestra** antes de enviarse, o sea que el servidor solo guarda datos cifrados

En mi homelab Vaultwarden funciona con Docker dentro del contenedor 102 y solo escucha dentro de ese contenedor (`127.0.0.1:8080`), y luego **Tailscale**, que está en el mismo contenedor, lo publica con un certificado HTTPS válido en una dirección a la que solo se puede entrar desde los dispositivos que están en la tailnet



Figura 4 · Cómo llega un dispositivo hasta Vaultwarden

LO QUE NECESITA CADA DISPOSITIVO

La dirección de Vaultwarden solo existe dentro de la tailnet, así que para usarlo cada dispositivo tiene que tener la app de Tailscale instalada y activada, también cuando está en casa, y desde internet no se puede llegar a Vaultwarden

7.2 Ficha del contenedor

ID	`102`
Nombre de host	`vaultwarden`
Tipo	Contenedor LXC sin privilegios, con anidamiento y `keyctl`
Sistema	Debian 13 (plantilla `debian-13-standard_13.1-2_amd64.tar.zst`)
Recursos	1 núcleo · 1024 MiB de RAM · 512 MiB de swap · disco de 8 GiB en `local-lvm`
Red	`eth0` en `vmbr0` · `192.168.1.62/24` · puerta de enlace `192.168.1.1`
Dispositivos	`/dev/net/tun`, que lo necesita Tailscale
Arranque	Se enciende solo al arrancar el servidor, el segundo (orden 2)
Zona horaria	La misma que el servidor (`Europe/Madrid`)
Software	Docker CE con Compose · Tailscale · Vaultwarden
Datos	`/opt/vaultwarden/data`
Acceso	`https://vaultwarden.tailxxxxxx.ts.net`

7.3 Creación del contenedor

Lo creo con el botón **Crear CT** y estos son los valores que puse en cada pestaña

PESTAÑA	CAMPO	VALOR
General	CT ID	`102`
	Nombre de host	`vaultwarden`
	Contenedor sin privilegios	Sí
	Anidamiento	Sí
	Contraseña de root	*Puesta, pero no la apunto aquí*
Plantilla	Almacenamiento / Plantilla	`local / debian-13-standard_13.1-2`
Discos	Almacenamiento / Tamaño	`local-lvm` · 8 GiB
CPU	Núcleos	1
Memoria	Memoria / Swap	1024 MiB / 512 MiB
Red	Nombre / Puente	`eth0 / vmbr0`
	IPv4	Estático · `192.168.1.62/24`
	Puerta de enlace (IPv4)	`192.168.1.1`
	IPv6	Estático, vacío
DNS	Dominio y servidores	Vacíos (coge lo que tiene el servidor)
Confirmar	Iniciar después de creado	**No**

POR QUÉ 1 GB DE RAM Y POR QUÉ NO LO ENCIENDO TODAVÍA

En este contenedor van a la vez Docker, Tailscale y Vaultwarden, por eso le doy más memoria que a Pi-hole, y no lo enciendo al crearlo porque antes le tengo que dar los permisos del apartado siguiente

7.4 Ajustes desde Proxmox

Esto lo hago desde la **Shell de Proxmox** con el contenedor todavía apagado

Shell de Proxmox · root@pve

```
# pct set 102 --features keyctl=1,nesting=1
# pct set 102 --dev0 /dev/net/tun
# pct set 102 --timezone host
# pct set 102 --onboot 1 --startup order=2
# pct start 102
```

`--features keyctl=1,nesting=1`	Son los permisos que necesita Docker para poder funcionar dentro de un contenedor sin privilegios
`--dev0 /dev/net/tun`	Le da acceso a la tarjeta de red virtual que usa Tailscale
`--timezone host`	La misma zona horaria que el servidor
`--onboot 1 --startup order=2`	Que se encienda solo y el segundo, después de Pi-hole

En el router hice la reserva DHCP de `192.168.1.62` para la MAC del contenedor, que se ve en **102 → Red**

7.5 Preparación del sistema

Desde **102 → Consola** y con el usuario `root` hago esto

Consola del contenedor · root@vaultwarden

```
# apt update && apt full-upgrade -y
# apt install -y ca-certificates curl nano
# date
# ls -l /dev/net/tun
```

`date` enseña la hora de España y `ls -l /dev/net/tun` me confirma que el dispositivo de Tailscale está ahí

7.6 Instalación de Docker

Docker lo instalo desde su **repositorio oficial** para Debian y no desde el de Debian, para tener siempre la última versión, y todo esto en la consola del contenedor

Consola del contenedor · root@vaultwarden

```
# Clave con la que se firman los paquetes de Docker
# install -m 0755 -d /etc/apt/keyrings
# curl -fsSL https://download.docker.com/linux/debian/gpg -o /etc/apt/keyrings/docker.asc
# chmod a+r /etc/apt/keyrings/docker.asc

# Repositorio de Docker
# tee /etc/apt/sources.list.d/docker.sources <<EOF
Types: deb
URIs: https://download.docker.com/linux/debian
Suites: $(. /etc/os-release && echo "$VERSION_CODENAME")
Components: stable
Architectures: $(dpkg --print-architecture)
Signed-By: /etc/apt/keyrings/docker.asc
EOF

# Instalación y prueba
# apt update
# apt install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin
# docker run --rm hello-world
```

RESULTADO

La prueba `docker run --rm hello-world` sacó «Hello from Docker!» a la primera, o sea que Docker funciona bien dentro del contenedor

7.7 Tailscale y HTTPS

Preparación en el panel de Tailscale

Primero en `login.tailscale.com` hago dos cosas

1. En **Machines** borro la máquina de la instalación de antes (`docker`)
2. En **DNS** activo **MagicDNS** y **HTTPS Certificates**, que hacen falta para que Tailscale le pueda dar a cada máquina un nombre y un certificado HTTPS válido

Instalación en el contenedor

Luego instalo Tailscale en la consola del contenedor

```
●●● Consola del contenedor · root@vaultwarden
# curl -fsSL https://tailscale.com/install.sh -o install-tailscale.sh
# sh install-tailscale.sh
# tailscale up
```

`tailscale up` te da un enlace que abro en el navegador del PC para iniciar sesión con mi cuenta de Tailscale, y desde ese momento el contenedor sale en el panel como la máquina `vaultwarden`

LE QUITO LA CADUCIDAD A LA CLAVE

Tailscale por defecto te obliga a volver a iniciar sesión en cada máquina cada 180 días, y en un servidor eso haría que Vaultwarden dejara de funcionar sin avisar, así que en el panel, en la máquina `vaultwarden`, elegí **... → Disable key expiry**

Publicación con HTTPS

Y para publicarlo con HTTPS pongo esto en la consola del contenedor

```
●●● Consola del contenedor · root@vaultwarden
# tailscale serve --bg 8080
# tailscale serve status
```

SALIDA REAL

```
https://vaultwarden.tailxxxxxx.ts.net (tailnet only)
|-- / proxy http://127.0.0.1:8080
```

`--bg`	Deja la publicación puesta en segundo plano, y sigue ahí aunque se reinicie el contenedor
`tailnet only`	A la dirección solo se puede entrar desde dispositivos de la tailnet y no desde internet
`proxy http://127.0.0.1:8080`	Tailscale recibe las conexiones HTTPS y se las pasa a Vaultwarden dentro del contenedor

7.8 Instalación de Vaultwarden

Vaultwarden lo he montado con **Docker Compose**, así que lo primero es crear una carpeta para él en `/opt` y dentro el archivo `compose.yaml`, todo esto desde la consola del contenedor

Consola del contenedor · root@vaultwarden

```
# mkdir -p /opt/vaultwarden
# cd /opt/vaultwarden
# nano compose.yaml
```

Y dentro del archivo he puesto esto

/opt/vaultwarden/compose.yaml

```
services:
  vaultwarden:
    image: vaultwarden/server:latest
    container_name: vaultwarden
    restart: unless-stopped
    environment:
      DOMAIN: "https://vaultwarden.tailxxxxxx.ts.net"
      SIGNUPS_ALLOWED: "true"
    volumes:
      - ./data:/data
    ports:
      - "127.0.0.1:8080:80"
```

`image`	La imagen oficial de Vaultwarden en la última versión
`restart: unless-stopped`	Se vuelve a encender solo si se cae o si reinicio el contenedor, menos si lo paro yo a mano
`DOMAIN`	La dirección con la que entro a Vaultwarden, que la necesita para que los enlaces salgan bien
`SIGNUPS_ALLOWED`	Deja crear cuentas, y lo pongo en `"true"` solo para poder crearme la mía (apartado 7.9)
`./data:/data`	Los datos se guardan fuera de Docker, en `/opt/vaultwarden/data`, así que aunque borre el contenedor de Docker no pierdo nada
`127.0.0.1:8080:80`	Vaultwarden solo escucha dentro del contenedor, es decir que la única forma de entrar es por Tailscale y siempre con HTTPS

Después ya lo arranco y miro que esté funcionando

Consola del contenedor · root@vaultwarden

```
# docker compose up -d
# docker compose ps
```

SALIDA REAL (DOCKER COMPOSE PS)

```
vaultwarden   vaultwarden/server:latest   "/start.sh"   vaultwarden   1 hours ago   Up 1 hours (healthy)   127.0.0.1:8080->80/tcp
```

Si pone **healthy** es que Vaultwarden ha pasado su propia comprobación y está todo bien, y el panel de administración no lo he activado porque no he puesto ningún `ADMIN_TOKEN`, así que por ahí no hay nada que tocar

7.9 Cuenta e importación

1. **Crear la cuenta**, con Tailscale activado en el PC abro `https://vaultwarden.tailxxxxxx.ts.net` y me creo la cuenta
2. **Importar las contraseñas**, en **Herramientas → Importar datos** con el formato **Bitwarden (json)** elijo el archivo que exporté antes de reinstalar (apartado 2.1) y pongo la contraseña del archivo
3. **Comprobar**, todo se importó bien
4. **Cerrar los registros**, para que nadie más se pueda crear una cuenta cambio `SIGNUPS_ALLOWED` a `"false"` en `compose.yaml` y vuelvo a aplicar el cambio

Cambio en compose.yaml · aplicar con docker compose

```
      SIGNUPS_ALLOWED: "false"

# docker compose up -d
```

LA CONTRASEÑA MAESTRA

La contraseña maestra no se puede recuperar, y si se te olvida no hay forma de abrir la caja fuerte, ni siquiera entrando al servidor

7.10 Aplicaciones de Bitwarden

Vaultwarden lo tengo puesto en el **móvil** y en el **PC**, y en cada app hice esto

1. Cierro la sesión del servidor de antes
2. En la pantalla de iniciar sesión elijo **Autoalojado** (self-hosted)

3. Pongo la dirección `https://vaultwarden.tailxxxxxx.ts.net` e inicio sesión

7.11 Copia de seguridad provisional

Hasta que tenga un disco externo para las copias automáticas, hago una copia a mano del contenedor y la guardo en el PC

Copia en Proxmox

Se hace en **102 → Copia de seguridad → Hacer copia ahora**

Almacenamiento	`local`
Modo	Instantánea (no hace falta apagar el contenedor)
Compresión	ZSTD

Aviso de SSH después de reinstalar

Cuando intenté bajar la copia con `scp`, SSH no me dejó conectar y me salió esto

SALIDA REAL (FRAGMENTO)

```
@    WARNING: REMOTE HOST IDENTIFICATION HAS CHANGED!     @
The fingerprint for the ED25519 key sent by the remote host is
SHA256:XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX.
Host key verification failed.
```

Cada servidor tiene unas **claves de host** que lo identifican y mi PC se acordaba de las del Proxmox de antes, entonces al reinstalar se crearon claves nuevas y SSH avisa porque no sabe si es eso o si alguien se está haciendo pasar por el servidor, así que antes de aceptar las nuevas comprobé que la huella era la de mi servidor de verdad desde la Shell de Proxmox

Shell de Proxmox · root@pve

```
# ssh-keygen -lf /etc/ssh/ssh_host_ed25519_key.pub
```

SALIDA REAL

```
256 SHA256:XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX root@pve (ED25519)
```

La huella es la misma que la del aviso, o sea que es mi servidor, así que borré la clave vieja del PC y bajé la copia a la carpeta de Descargas con PowerShell, aceptando la huella nueva al conectar

PowerShell · PC con Windows

```
PS> ssh-keygen -R 192.168.1.60
PS> scp root@192.168.1.60:/var/lib/vz/dump/vzdump-lxc-102-*.tar.zst "$HOME\Downloads\"
```

Archivo	vzdump-lxc-102-2026_09_23-21_35_14.tar.zst
Tamaño	≈ 608 MB
Dónde está	En la carpeta de Descargas del PC

7.12 Resumen

QUÉ HE COMPROBADO	RESULTADO
El contenedor 102 creado con los valores que tenía pensados	Correcto
Docker funcionando (`hello-world` a la primera)	Correcto
Vaultwarden funcionando y en estado `healthy`	Correcto
Publicado con HTTPS solo dentro de la tailnet	Correcto
Quitada la caducidad de la clave de Tailscale	Configurado
Contraseñas importadas y registros cerrados	Correcto
Las apps puestas en el móvil y en el PC	Correcto
Copia de seguridad bajada al PC	Correcto
Reserva DHCP de la `.62` en el router	Configurado

SIGUIENTE PASO

Capítulo 8 · Tailscale · LXC 103 · 192.168.1.66

Entrar a toda la red de casa desde fuera y con las mismas IPs

CAPÍTULO 08 · SERVICIO

Tailscale (acceso remoto)

Entrar a toda la red de casa desde fuera y con las mismas IPs, gracias a un contenedor que hace de subnet router

8.1 Qué es y cómo funciona

Un **subnet router** es una máquina de la tailnet que te deja entrar a una red entera y no solo a ella misma, entonces el contenedor 103 le dice a la tailnet que pasando por él se puede llegar a la red de casa, y así con la app de Tailscale activada el móvil puede abrir `http://192.168.1.61/admin` o `https://192.168.1.60:8006` desde fuera como si estuviera en casa, y los marcadores me sirven igual dentro que fuera

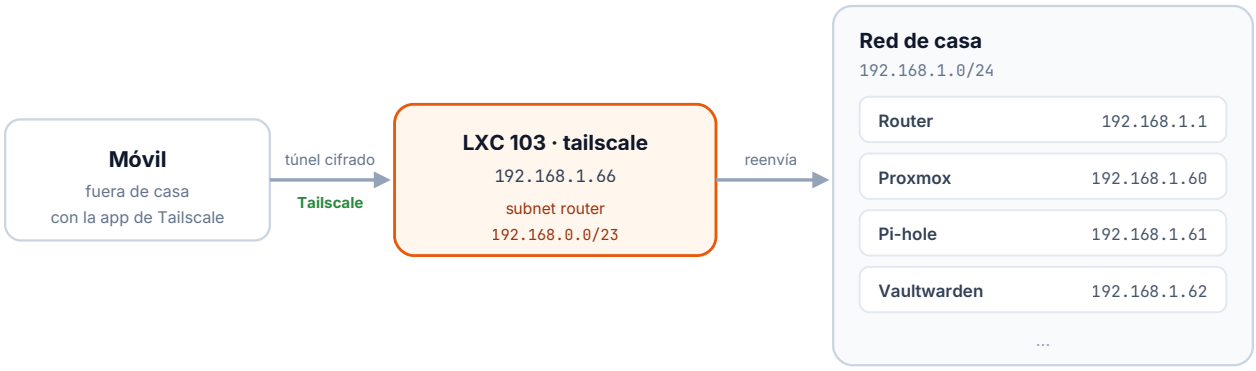


Figura 5 · Entrar a la red de casa desde fuera pasando por el subnet router

Por qué anuncio 192.168.0.0/23 y no 192.168.1.0/24

La red de casa es `192.168.1.0/24`, pero si anunciara esa misma red, un dispositivo que está en casa con Tailscale activado tendría dos caminos iguales para llegar a `192.168.1.x` y el sistema podría mandar el tráfico por Tailscale en vez de directo, y en el peor de los casos el dispositivo se queda sin DNS y sin internet

Por eso anuncio `192.168.0.0/23`, que es una red un poco más grande que tiene dentro la de casa, ya que los sistemas operativos siempre eligen la ruta **más concreta** (la que tiene el número más alto después de la barra)

DÓNDE ESTÁ EL DISPOSITIVO	CAMINOS HACIA 192.168.1.X	CUÁL GANA	QUÉ PASA
En casa	Red local `/24` y Tailscale `/23`	La `/24`, que es más concreta	El tráfico va directo, sin pasar por Tailscale
Fuera de casa	Solo Tailscale `/23`	La `/23`	El tráfico pasa por el contenedor 103

LO QUE NO FUNCIONA

Si el dispositivo está en el WiFi de otra casa que también use `192.168.1.x` (pasa mucho con los routers de Movistar), gana la red de esa casa y no se llega a las IPs del homelab, pero con datos móviles no pasa, y Vaultwarden funciona siempre porque usa su propia dirección de Tailscale

8.2 Ficha del contenedor

ID	`103`
Nombre de host	`tailscale`
Tipo	Contenedor LXC sin privilegios y con anidamiento
Sistema	Debian 13 (plantilla `debian-13-standard_13.1-2_amd64.tar.zst`)
Recursos	1 núcleo · 512 MiB de RAM · 512 MiB de swap · disco de 8 GiB en `local-lvm`
Red	`eth0` en `vmbr0` · `192.168.1.66/24` · puerta de enlace `192.168.1.1`
Dispositivos	`/dev/net/tun`, que lo necesita Tailscale
Arranque	Se enciende solo al arrancar el servidor, el tercero (orden 3)
Zona horaria	La misma que el servidor (`Europe/Madrid`)
Ruta anunciada	`192.168.0.0/23`
IP en la tailnet	`100.64.0.1`

8.3 Creación del contenedor

Lo creo con el botón **Crear CT** y estos son los valores que puse en cada pestaña

PESTAÑA	CAMPO	VALOR
General	Nodo	`pve`
	CT ID	`103`
	Nombre de host	`tailscale`
	Contenedor sin privilegios	Sí
	Anidamiento	Sí
	Contraseña de root	*Puesta, pero no la apunto aquí*
	Clave SSH pública	Vacío
Plantilla	Almacenamiento / Plantilla	`local / debian-13-standard_13.1-2`
Discos	Almacenamiento / Tamaño	`local-lvm` · 8 GiB
CPU	Núcleos	1
Memoria	Memoria / Swap	512 MiB / 512 MiB
Red	Nombre / Puente	`eth0 / vmbr0`
	Etiqueta VLAN / Cortafuegos	Vacío / como viene (marcado)
	IPv4	Estático · `192.168.1.66/24`
	Puerta de enlace (IPv4)	`192.168.1.1`
	IPv6	Estático, vacío
DNS	Dominio y servidores	Vacíos (coge lo que tiene el servidor)
Confirmar	Iniciar después de creado	**No**

8.4 Ajustes desde Proxmox

Esto lo hago desde la **Shell de Proxmox** con el contenedor todavía apagado

Shell de Proxmox · root@pve

```
# pct set 103 --dev0 /dev/net/tun
# pct set 103 --timezone host
# pct set 103 --onboot 1 --startup order=3
# pct start 103
```

`--dev0 /dev/net/tun`	Le da acceso a la tarjeta de red virtual que usa Tailscale
`--timezone host`	La misma zona horaria que el servidor
`--onboot 1 --startup order=3`	Que se encienda solo y el tercero, después de Pi-hole y Vaultwarden

En el router hice la reserva DHCP de `192.168.1.66` para la MAC del contenedor (`BC:24:11:XX:XX:XX`)

8.5 Preparación del sistema

Desde **103 → Consola** y con el usuario `root` hago esto

Consola del contenedor · root@tailscale

```
# apt update && apt full-upgrade -y
# apt install -y curl
# date
# ls -l /dev/net/tun
```

8.6 Reenvío de tráfico

Para hacer de puente entre la tailnet y la red de casa el contenedor tiene que poder **pasar paquetes** de una red a otra, igual que un router, pero Linux por defecto no lo hace, así que lo activo en la consola del contenedor

Consola del contenedor · root@tailscale

```
# echo 'net.ipv4.ip_forward = 1' | tee -a /etc/sysctl.d/99-tailscale.conf
# echo 'net.ipv6.conf.all.forwarding = 1' | tee -a /etc/sysctl.d/99-tailscale.conf
# sysctl -p /etc/sysctl.d/99-tailscale.conf
```

`net.ipv4.ip_forward = 1`	Activa el reenvío de paquetes en IPv4
`net.ipv6.conf.all.forwarding = 1`	Lo mismo pero en IPv6, que Tailscale lo recomienda aunque la red de casa sea IPv4
`/etc/sysctl.d/99-tailscale.conf`	Al guardarlo en este archivo el cambio se queda aunque reinicie
`sysctl -p`	Aplica el archivo en ese momento sin tener que reiniciar

8.7 Instalación de Tailscale

Luego instalo Tailscale también desde la consola del contenedor

Consola del contenedor · root@tailscale

```
# curl -fsSL https://tailscale.com/install.sh -o install-tailscale.sh
# sh install-tailscale.sh
# tailscale up --advertise-routes=192.168.0.0/23
```

`--advertise-routes` sirve para decir qué red le ofrezco a la tailnet, y `tailscale up` te da un enlace que abro en el navegador del PC para iniciar sesión con mi cuenta de Tailscale

8.8 Configuración en el panel

En `login.tailscale.com` → **Machines**, en el menú **...** de la máquina `tailscale`, hago esto

1. **Edit route settings**, marco la ruta `192.168.0.0/23` y guardo, porque Tailscale no activa ninguna ruta anunciada hasta que la apruebas aquí
2. **Disable key expiry**, igual que con Vaultwarden, para que el contenedor no pierda la sesión a los 180 días

EL PC DE CASA

En la app de Tailscale del PC dejo sin marcar la opción **Use Tailscale subnets**, ya que el PC siempre está en casa y no le hace falta usar el subnet router

DNS DE LA TAILNET

Más adelante, en la parte de **DNS** de este mismo panel, puse Pi-hole (`192.168.1.61`) como DNS de toda la tailnet para que también bloquee anuncios fuera de casa, y las consultas de los dispositivos que están fuera llegan a Pi-hole pasando por este contenedor, todo esto lo explico en el apartado 6.15

8.9 Verificación

Para ver que está todo bien miro el estado de Tailscale en la consola del contenedor

Consola del contenedor · root@tailscale

```
# tailscale status
```

SALIDA REAL (IPS Y NOMBRES DE EJEMPLO)

```
100.64.0.1      tailscale        usuario@  linux    -
100.64.0.2      pc-windows       usuario@  windows  -
100.64.0.3      iphone-13        usuario@  iOS      idle, tx 8069492 rx 460148
100.64.0.4      portatil         usuario@  linux    offline, last seen 1d ago
100.64.0.5      vaultwarden      usuario@  linux    -
```

El contenedor sale en la tailnet con la IP `100.64.0.1`, junto con el resto de mis dispositivos

Pruebas de acceso

PRUEBA	QUÉ ABRÍ	RESULTADO
Móvil con **datos móviles** y Tailscale activado	Pi-hole en `http://192.168.1.61/admin`	Correcto
Móvil con **datos móviles** y Tailscale activado	Proxmox en `https://192.168.1.60:8006`	Correcto
Móvil **en casa** con Tailscale activado	Internet y los servicios de casa	Correcto

Todo cargó a la primera y sin cambiar nada en la app de Tailscale del móvil

8.10 Resumen

QUÉ HE COMPROBADO	RESULTADO
El contenedor 103 creado con los valores que tenía pensados y la IP `192.168.1.66`	Correcto
Reenvío de tráfico activado	Correcto
Tailscale instalado y anunciando `192.168.0.0/23`	Correcto
Ruta aprobada y quitada la caducidad de la clave	Configurado
Se puede entrar a las IPs del homelab desde fuera de casa	Correcto
Los dispositivos que están en casa con Tailscale activado funcionan normal	Correcto
Reserva DHCP de la `.66` en el router	Configurado

SIGUIENTE PASO

Capítulo 9 · Uptime Kuma · LXC 104 · 192.168.1.64

Vigilar que todos los servicios del homelab estén funcionando

CAPÍTULO 09 · SERVICIO

Uptime Kuma

Vigilar todos los servicios del homelab y que me llegue un correo cuando alguno deja de funcionar

9.1 Qué es y cómo funciona

Uptime Kuma mira cada 60 segundos que todos los servicios contesten, y dependiendo del servicio lo hace con un ping, abriendo su página web o haciendo una consulta DNS, entonces si alguno deja de contestar me manda un correo y cuando vuelve me manda otro, y todo se puede ver en su panel web en `http://192.168.1.64:3001`

Lo he instalado directamente con **Node.js**, sin Docker, y el contenedor también lleva **Tailscale** porque a Vaultwarden solo se puede llegar por su dirección de Tailscale, y así lo puede comprobar de verdad, incluido su certificado HTTPS

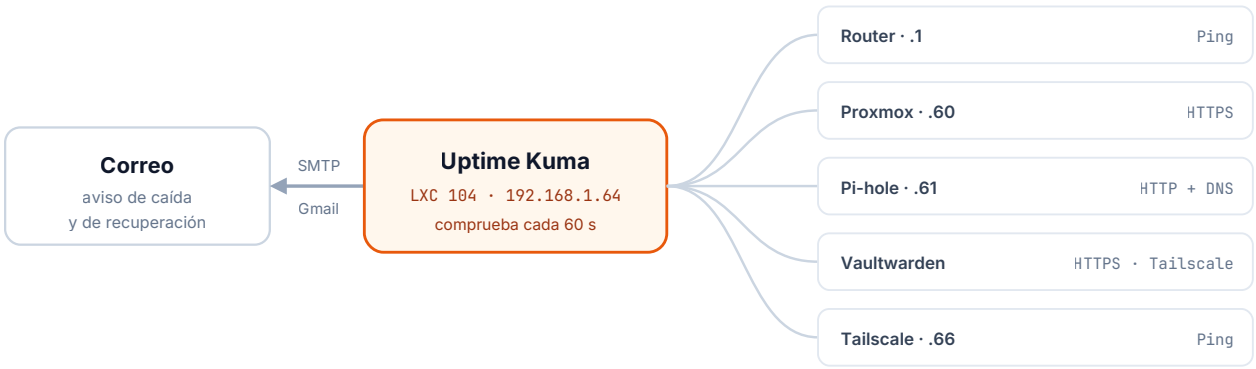


Figura 6 · Qué comprueba Uptime Kuma y cómo me avisa

LO QUE NO PUEDE VIGILAR

Uptime Kuma está en el mismo servidor que los servicios que vigila, así que me avisa si se cae un servicio pero no puede avisarme si se apaga el Chromebox entero o si se va internet en casa

9.2 Ficha del contenedor

ID	104
Nombre de host	uptime-kuma
Tipo	Contenedor LXC sin privilegios y con anidamiento
Sistema	Debian 13 (plantilla `debian-13-standard_13.1-2_amd64.tar.zst`)
Recursos	1 núcleo · 1024 MiB de RAM · 512 MiB de swap · disco de 8 GiB en `local-lvm`
Red	`eth0` en `vmbr0` · `192.168.1.64/24` · puerta de enlace `192.168.1.1`
Dispositivos	`/dev/net/tun`, que lo necesita Tailscale
Arranque	Se enciende solo al arrancar el servidor, el cuarto (orden 4)
Zona horaria	La misma que el servidor (`Europe/Madrid`)
Software	Node.js 20.19 (el paquete de Debian 13) · Uptime Kuma 2 · Tailscale
Instalación	En `/opt/uptime-kuma`, como servicio `uptime-kuma` de systemd
Base de datos	SQLite
Panel web	http://192.168.1.64:3001
Avisos	Por correo con Gmail (SMTP, puerto 587)

9.3 Comprobación previa de la IP

Antes de crear el contenedor miro que ningún otro dispositivo de la red esté usando la `192.168.1.64`, y para eso uso `arping` desde la Shell de Proxmox, que pregunta directamente en la red quién tiene esa IP, y a diferencia del ping aquí ningún dispositivo puede dejar de contestar

Shell de Proxmox · root@pve

```
# arping -c 3 -I vmbr0 192.168.1.64
```

No contestó **nadie**, o sea que la IP está libre

9.4 Creación del contenedor

Lo creo con el botón **Crear CT** y estos son los valores que puse en cada pestaña

PESTAÑA	CAMPO	VALOR
General	Nodo	`pve`
	CT ID	`104`
	Nombre de host	`uptime-kuma`
	Contenedor sin privilegios	Sí
	Anidamiento	Sí
	Contraseña de root	*Puesta, pero no la apunto aquí*
	Clave SSH pública	Vacío
Plantilla	Almacenamiento / Plantilla	`local / debian-13-standard_13.1-2`
Discos	Almacenamiento / Tamaño	`local-lvm` · 8 GiB
CPU	Núcleos	1
Memoria	Memoria / Swap	1024 MiB / 512 MiB
Red	Nombre / Puente	`eth0 / vmbr0`
	Etiqueta VLAN / Cortafuegos	Vacío / como viene (marcado)
	IPv4	Estático · `192.168.1.64/24`
	Puerta de enlace (IPv4)	`192.168.1.1`
	IPv6	Estático, vacío
DNS	Dominio y servidores	Vacíos (coge lo que tiene el servidor)
Confirmar	Iniciar después de creado	**No**

POR QUÉ 1 GB DE RAM

Uptime Kuma gasta poco cuando ya está funcionando, pero al instalarlo con `npm` necesita bastante memoria durante unos minutos

9.5 Ajustes desde Proxmox

Esto lo hago desde la Shell de Proxmox con el contenedor todavía apagado

Shell de Proxmox · root@pve

```
# pct set 104 --dev0 /dev/net/tun
# pct set 104 --timezone host
# pct set 104 --onboot 1 --startup order=4
# pct start 104
```

Lo de `/dev/net/tun` es para Tailscale, y el contenedor arranca el cuarto para que cuando se encienda los servicios que vigila ya estén funcionando, y en el router hice la reserva DHCP de `192.168.1.64` para la MAC del contenedor

9.6 Preparación del sistema

En la consola del contenedor actualizo el sistema e instalo lo que hace falta

Consola del contenedor · root@uptime-kuma

```
# apt update && apt full-upgrade -y
# apt install -y curl git nodejs npm
# date
# node --version
# ping -c 10 1.1.1.1
```

`git`	Para bajar el código de Uptime Kuma
`nodejs / npm`	Es con lo que funciona Uptime Kuma y su gestor de paquetes, y Uptime Kuma 2 necesita Node.js 20.4 o más, y Debian 13 trae la 20.19
`ping -c 10 1.1.1.1`	Para ver que internet va estable (0 % de pérdida) antes de instalar nada

9.7 Instalación de Uptime Kuma

Luego bajo Uptime Kuma y lo preparo, también en la consola del contenedor

Consola del contenedor · root@uptime-kuma

```
# git clone https://github.com/louislam/uptime-kuma.git /opt/uptime-kuma
# cd /opt/uptime-kuma
# npm run setup
```

`git clone` baja el proyecto oficial en `/opt/uptime-kuma` y `npm run setup` elige la última versión estable, instala lo que necesita y baja la web ya preparada, y en este servidor tarda unos cuantos minutos, y los avisos `npm WARN` que salen mientras tanto son normales

9.8 Servicio de arranque automático

Para que Uptime Kuma se encienda solo con el contenedor y se reinicie si falla, le creo un servicio de **systemd** con este archivo

/etc/systemd/system/uptime-kuma.service

```
[Unit]
Description=Uptime Kuma
After=network-online.target
Wants=network-online.target

[Service]
Type=simple
WorkingDirectory=/opt/uptime-kuma
ExecStart=/usr/bin/node server/server.js
Restart=on-failure
RestartSec=5

[Install]
WantedBy=multi-user.target
```

`After / Wants`	Espera a que la red esté lista antes de arrancar
`WorkingDirectory`	La carpeta desde donde se ejecuta Uptime Kuma
`ExecStart`	El comando que arranca Uptime Kuma con Node.js
`Restart=on-failure`	Si se cierra por un error, systemd lo vuelve a arrancar a los 5 segundos
`WantedBy=multi-user.target`	Hace que se encienda solo cada vez que arranca el contenedor

Y después activo el servicio

```
Consola del contenedor · root@uptime-kuma

# systemctl daemon-reload
# systemctl enable --now uptime-kuma
# systemctl status uptime-kuma
# ss -tlnp | grep 3001
```

El servicio se quedó en **active (running)** y escuchando en el puerto `3001`

9.9 Primer acceso

1. Desde el PC abro `http://192.168.1.64:3001`
2. Como base de datos elijo **SQLite**, que es un solo archivo, no necesita otro servidor y para un homelab va de sobra
3. Creo el usuario administrador del panel *(los datos no los apunto aquí)*
4. El idioma se puede cambiar en **Ajustes → Apariencia → Idioma**

9.10 Tailscale en el contenedor

Para que Uptime Kuma pueda comprobar Vaultwarden por su dirección de verdad, meto el contenedor en la tailnet desde su consola

```
Consola del contenedor · root@uptime-kuma

# curl -fsSL https://tailscale.com/install.sh -o install-tailscale.sh
# sh install-tailscale.sh
# tailscale up
```

Después de iniciar sesión, en el panel de Tailscale le quité la caducidad a la clave de la máquina `uptime-kuma` (**... → Disable key expiry**), igual que en los contenedores 102 y 103

9.11 Monitores

Los creé con **Añadir nuevo monitor**, todos para que comprueben cada 60 segundos

NOMBRE	TIPO	DIRECCIÓN O DATOS	NOTAS
Router	Ping	`192.168.1.1`	
Proxmox	HTTP(s)	`https://192.168.1.60:8006`	Con «Ignorar errores TLS/SSL», porque el certificado lo hace Proxmox
Pi-hole (web)	HTTP(s)	`http://192.168.1.61/admin/`	
Pi-hole (DNS)	DNS	Nombre `google.com` · Servidor `192.168.1.61` · Puerto `53`	Mira que Pi-hole resuelva nombres
Vaultwarden	HTTP(s)	`https://vaultwarden.tailxxxxxx.ts.net/alive`	También avisa si va a caducar el certificado
Tailscale (subnet router)	Ping	`192.168.1.66`	

> **TIPO DE MONITOR**
> El campo cambia según el tipo que elijas, en los HTTP(s) pone **URL** y en Ping y DNS pone **Hostname**, y en el monitor DNS el *Hostname* es el nombre que quieres resolver (`google.com`) y la IP de Pi-hole va en *Servidor de resolución*

9.12 Avisos por correo

Contraseña de aplicación de Google

Gmail no deja que un programa mande correos con la contraseña normal de la cuenta, así que hace falta una **contraseña de aplicación**, que solo sirve para esto y se puede borrar cuando quieras sin que le pase nada a la cuenta

1. La **verificación en dos pasos** de la cuenta de Google tiene que estar activada
2. En `myaccount.google.com` busco **Contraseñas de aplicaciones**
3. Creo una que se llame `Uptime Kuma` y copio la contraseña de 16 letras, que solo te la enseña una vez

Configuración en Uptime Kuma

Esto va en **Ajustes → Notificaciones → Configurar notificación**

Tipo de notificación	Correo electrónico (SMTP)
Nombre descriptivo	`Correo`
Nombre de host	`smtp.gmail.com`
Puerto	587
Seguridad	Ninguna / STARTTLS (25, 587)
Ignorar error TLS	No
Usuario	*Mi dirección de Gmail (no la apunto aquí)*
Contraseña	*La contraseña de aplicación (no la apunto aquí)*
Remitente / destinatario	*Mis correos (no los apunto aquí)*
Activado por defecto	Sí, así los monitores nuevos ya lo tienen activado
Aplicar en todos los monitores existentes	Sí

Le di al botón **Probar**, me llegó el correo de prueba y guardé la notificación, y el aviso se puede cambiar o ampliar cuando quiera desde la misma pantalla, por ejemplo añadiendo Telegram

9.13 Verificación

Para comprobar que Uptime Kuma de verdad detecta las caídas y avisa, paré Vaultwarden a propósito desde la consola del contenedor 102 y luego lo volví a arrancar

Consola del contenedor 102 · root@vaultwarden

```
# cd /opt/vaultwarden
# docker compose stop
  … esperar uno o dos minutos …
# docker compose start
```

MOMENTO	EN EL PANEL	CORREO
Vaultwarden parado	El monitor se pone en **Caído**	Me llega el aviso de caída
Vaultwarden arrancado otra vez	El monitor vuelve a **Activo**	Me llega el aviso de que ha vuelto

9.14 Resumen

QUÉ HE COMPROBADO	RESULTADO
La IP `192.168.1.64` estaba libre antes de crear el contenedor	Correcto
El contenedor 104 creado con los valores que tenía pensados	Correcto
Node.js 20.4 o más e internet sin pérdidas	Correcto
Uptime Kuma instalado como servicio y escuchando en el puerto 3001	Correcto
Tailscale instalado y quitada la caducidad de la clave	Configurado
Los seis monitores creados y todos activos	Correcto
Me llegaron los correos de prueba, de caída y de recuperación	Correcto
Reserva DHCP de la `.64` en el router	Configurado

SIGUIENTE PASO

Capítulo 10 · Jellyfin · LXC 105 · 192.168.1.65

Mi servidor de películas, que convierte el vídeo con la gráfica

CAPÍTULO 10 · SERVICIO

Jellyfin

Mi propio servidor para ver películas y series en cualquier dispositivo, que convierte el vídeo con la gráfica Intel

10.1 Qué es y cómo funciona

Jellyfin ordena las películas y las series, les baja la carátula y la sinopsis y las manda a cualquier dispositivo, ya sea el navegador, la tele o el móvil, y si algún dispositivo no puede reproducir el formato del archivo Jellyfin lo **convierte en el momento** (a esto se le llama transcodificar), y esa conversión la hace la gráfica Intel del servidor con **QuickSync** y no el procesador

Las películas se guardan en un volumen aparte de 100 GiB dentro del contenedor y las subo desde el PC por SFTP

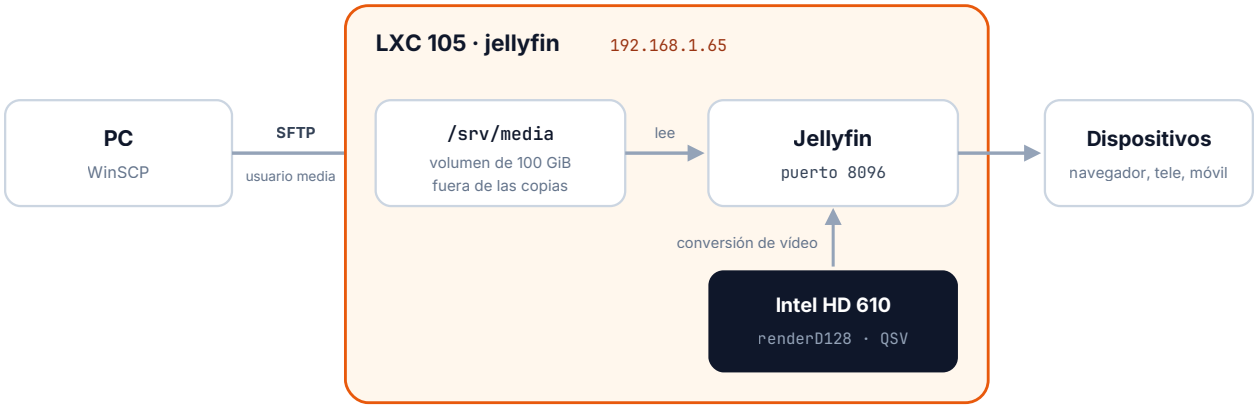


Figura 7 · Cómo subo, guardo y veo las películas

LO QUE DA DE SÍ LA GRÁFICA HD 610

Lo mejor es la **reproducción directa**, que es cuando el dispositivo reproduce el archivo tal cual y el servidor casi no trabaja, y cuando hay que convertir se encarga la gráfica, pero en la prueba (apartado 10.11) una sola conversión a 720p le ocupó entre el 70 % y el 90 % de su parte de vídeo, o sea que **una conversión a la vez va bien pero dos irían justas**, y además no puede pasar de HDR a SDR (*tone mapping*) porque lo que hace falta de Intel (OpenCL) para esta gráfica no existe en Debian 13, aunque las películas HDR se ven bien en los dispositivos que las reproducen directamente

10.2 Ficha del contenedor

ID	105
Nombre de host	jellyfin
Tipo	Contenedor LXC sin privilegios y con anidamiento
Sistema	Debian 13 (plantilla `debian-13-standard_13.1-2_amd64.tar.zst`)
Recursos	2 núcleos · 2048 MiB de RAM · 512 MiB de swap · disco de 16 GiB en `local-lvm`
Volumen de películas	`mp0` de 100 GiB en `local-lvm`, montado en `/srv/media` y sin copia de seguridad
Gráfica	`/dev/dri/renderD128`, con el grupo `video` (GID 44)
Red	`eth0` en `vmbr0` · `192.168.1.65/24` · puerta de enlace `192.168.1.1`
Arranque	Se enciende solo al arrancar el servidor, el quinto (orden 5)
Zona horaria	La misma que el servidor (`Europe/Madrid`)
Software	Jellyfin desde su repositorio oficial, con `jellyfin-ffmpeg`
Aceleración	Intel QuickSync (QSV)
Subida de archivos	SFTP con el usuario `media`
Panel web	`http://192.168.1.65:8096`

10.3 Comprobación previa de la IP

Igual que con Uptime Kuma, antes de nada miro desde la Shell de Proxmox que la IP no la use nadie

Shell de Proxmox · root@pve

```
# arping -c 3 -I vmbr0 192.168.1.65
```

No contestó **nadie**, así que la IP está libre

10.4 Creación del contenedor

Lo creo con el botón **Crear CT** y estos son los valores que puse en cada pestaña

PESTAÑA	CAMPO	VALOR
General	Nodo	`pve`
	CT ID	`105`
	Nombre de host	`jellyfin`
	Contenedor sin privilegios	Sí
	Anidamiento	Sí
	Contraseña de root	*Puesta, pero no la apunto aquí*
	Clave SSH pública	Vacío
Plantilla	Almacenamiento / Plantilla	`local / debian-13-standard_13.1-2`
Discos	Almacenamiento / Tamaño	`local-lvm` · 16 GiB
CPU	Núcleos	2
Memoria	Memoria / Swap	2048 MiB / 512 MiB
Red	Nombre / Puente	`eth0 / vmbr0`
	Etiqueta VLAN / Cortafuegos	Vacío / como viene (marcado)
	IPv4	Estático · `192.168.1.65/24`
	Puerta de enlace (IPv4)	`192.168.1.1`
	IPv6	Estático, vacío
DNS	Dominio y servidores	Vacíos (coge lo que tiene el servidor)
Confirmar	Iniciar después de creado	**No**

POR QUÉ TIENE MÁS RECURSOS QUE LOS DEMÁS

Jellyfin es el servicio que más pide, ya que usa 2 núcleos cuando convierte vídeo, 2 GB de RAM y 16 GiB de disco para su base de datos, las carátulas y los archivos temporales de las conversiones, y las películas no van en este disco sino en el volumen del apartado siguiente

10.5 Ajustes desde Proxmox

Esto lo hago desde la Shell de Proxmox con el contenedor todavía apagado

Shell de Proxmox · root@pve

```
# pct set 105 --mp0 local-lvm:100,mp=/srv/media,backup=0
# pct set 105 --dev0 /dev/dri/renderD128,gid=44
# pct set 105 --timezone host
# pct set 105 --onboot 1 --startup order=5
# pct start 105
```

`--mp0 local-lvm:100,mp=/srv/media,backup=0`	Crea un volumen nuevo de 100 GiB, lo monta en `/srv/media` y lo deja fuera de las copias de seguridad, porque las películas ocuparían muchísimo y se pueden volver a conseguir
`--dev0 /dev/dri/renderD128,gid=44`	Le pasa la gráfica al contenedor con el grupo `video` (que en Debian es el GID 44), y luego meto en ese grupo al usuario de Jellyfin
`--timezone host`	La misma zona horaria que el servidor
`--onboot 1 --startup order=5`	Que se encienda solo y el último de todos

En el router hice la reserva DHCP de `192.168.1.65` para la MAC del contenedor

10.6 Preparación del sistema

En la consola del contenedor actualizo el sistema y miro que esté todo bien

Consola del contenedor · root@jellyfin

```
# apt update && apt full-upgrade -y
# apt install -y curl
# date
# ping -c 10 1.1.1.1
# ls -l /dev/dri
# df -h /srv/media
```

Internet va sin pérdidas, el dispositivo `renderD128` está ahí con el grupo `video` y el volumen de películas está montado en `/srv/media` con unos 98 GiB para usar, y lo que falta hasta 100 lo ocupa el propio sistema de archivos

10.7 Instalación de Jellyfin

Uso el instalador oficial, que añade el repositorio de Jellyfin e instala el servidor, la web y `jellyfin-ffmpeg`, y este último ya trae el driver de vídeo de Intel (iHD), así que no hace falta instalar drivers aparte

Consola del contenedor · root@jellyfin

```
# curl -fsSL https://repo.jellyfin.org/install-debuntu.sh -o install-jellyfin.sh
# bash install-jellyfin.sh
```

Después le doy permiso a Jellyfin para usar la gráfica y compruebo que la reconoce

Consola del contenedor · root@jellyfin

```
# usermod -aG video jellyfin
# systemctl restart jellyfin
# systemctl status jellyfin
# /usr/lib/jellyfin-ffmpeg/vainfo --display drm --device /dev/dri/renderD128
```

`usermod -aG video jellyfin`	Mete al usuario del servicio en el grupo `video`, que es el que puede usar la gráfica
`vainfo`	Enseña el driver que está cargado y los formatos que puede procesar la gráfica, y me salió el **driver iHD de Intel** con su lista de formatos

10.8 Carpetas y subida de películas

Luego preparo el SFTP, el usuario y las carpetas en la consola del contenedor

Consola del contenedor · root@jellyfin

```
# apt install -y openssh-server
# adduser media
# mkdir -p /srv/media/peliculas /srv/media/series
# chown -R media:media /srv/media/peliculas /srv/media/series
```

Las películas las sube un usuario normal (`media`) y no `root`, ya que Jellyfin solo tiene que poder leerlas

Subida desde el PC con WinSCP

Protocolo	SFTP
Servidor / puerto	`192.168.1.65` · `22`
Usuario	`media`
Destino	`/srv/media/peliculas` o `/srv/media/series`

Nombres de archivo

Para que Jellyfin reconozca cada título y le baje la carátula y la sinopsis hay que seguir esta estructura

TIPO	RUTA DE EJEMPLO
Película	`peliculas/Titulo (Año)/Titulo (Año).mkv`
Serie	`series/Nombre Serie/Season 01/Nombre Serie S01E01.mkv`

10.9 Asistente inicial

En `http://192.168.1.65:8096` elijo el idioma **Español**, creo el usuario administrador *(los datos no los apunto aquí)* y hago dos bibliotecas

BIBLIOTECA	CARPETA	METADATOS
Películas	`/srv/media/peliculas`	Idioma Español · País España
Programas	`/srv/media/series`	Idioma Español · País España

Las demás pantallas del asistente las dejé como venían

10.10 Aceleración por hardware

Esto está en **Panel de control → Reproducción → Transcodificación**

OPCIÓN	VALOR
Aceleración por hardware	**Intel QuickSync (QSV)**
Dispositivo QSV	`/dev/dri/renderD128`
Decodificación por hardware activada para	H264, HEVC, MPEG2, VC1, VP8, VP9, HEVC 10bit, VP9 10bit
Decodificación desactivada para	AV1 y HEVC REext, porque la HD 610 no los soporta
Codificadores de bajo consumo (H.264 y HEVC)	Desactivados
Permitir codificación en HEVC	Desactivado, para que funcione en todos los dispositivos
Mapeo de tonos (tone mapping)	Desactivado, porque con esta gráfica en Debian 13 no se puede

10.11 Verificación

Puse una película en el navegador obligando a que la convierta (**Ajustes → Calidad → 720p – 2 Mbps** en el reproductor), y mientras tanto miré cuánto se usaba la gráfica desde la Shell de Proxmox

```
Shell de Proxmox · root@pve
# apt install -y intel-gpu-tools
# intel_gpu_top
```

PRUEBA	RESULTADO	ESTADO
Subir una película por SFTP	La reconoció con su carátula y su sinopsis a la primera	Correcto
Conversión a 720p – 2 Mbps	La fila **Video** de `intel_gpu_top` estaba entre el **70 % y el 90 %**, o sea que la conversión la hace la gráfica	Correcto
Monitor en Uptime Kuma	HTTP(s) a `http://192.168.1.65:8096/health`, en verde	Correcto

10.12 Resumen

QUÉ HE COMPROBADO	RESULTADO
La IP `192.168.1.65` estaba libre y el contenedor 105 creado con los valores que tenía pensados	Correcto
El volumen de películas de 100 GiB montado y fuera de las copias	Correcto
La gráfica llega al contenedor y `vainfo` la reconoce (driver iHD)	Correcto
Jellyfin instalado desde el repositorio oficial y funcionando	Correcto
Subida de películas por SFTP con el usuario `media`	Correcto
Conversión de vídeo con la gráfica usando QuickSync	Correcto
Monitor de Jellyfin en Uptime Kuma	Configurado
Reserva DHCP de la `.65` en el router	Configurado

SIGUIENTE PASO

Copias de seguridad

Copias automáticas de todos los contenedores en un disco externo cuando lo tenga, y después pasar las películas a ese disco

ANEXO A

Glosario

Las palabras técnicas que salen en el documento, explicadas de forma sencilla

Proxmox VE	Un sistema basado en Debian para crear y manejar máquinas virtuales y contenedores desde una página web
Máquina virtual (VM)	Un ordenador entero simulado, con su propio sistema operativo y su propio kernel
Contenedor LXC	Un Linux aislado que usa el mismo kernel que el servidor, gasta mucho menos que una VM pero tiene su propia IP y sus propios programas
Plantilla (CT)	La imagen base de un sistema operativo que se usa para crear un contenedor
Contenedor sin privilegios	Un contenedor donde el `root` de dentro no manda sobre el servidor, y por eso es más seguro
Anidamiento	Una opción del contenedor (en inglés *nesting*) que deja que dentro funcionen algunas cosas modernas de Linux y, si hace falta, otros contenedores
pct	La herramienta de Proxmox para manejar los contenedores desde la terminal
Kernel	El núcleo del sistema operativo, la parte que controla el hardware
ISO	Una imagen de un disco de instalación metida en un solo archivo
UEFI	El firmware moderno que arranca el equipo, el que ha sustituido a la BIOS de toda la vida
Hash SHA-256	Es como la huella de un archivo, y si cambia aunque sea un poco del archivo la huella sale totalmente distinta
ext4	El sistema de archivos normal de Linux
ZFS	Un sistema de archivos más avanzado, pensado sobre todo para cuando tienes varios discos, y gasta más RAM
LVM	Un sistema que divide un disco en partes (volúmenes) a las que se les puede cambiar el tamaño
LVM-thin	Un tipo de LVM donde cada volumen solo ocupa lo que usa de verdad
Swap	Una parte del disco que se usa como memoria de reserva cuando la RAM se llena
FQDN	El nombre completo de un equipo con el dominio incluido, por ejemplo `pve.home.arpa`
CIDR (/24)	Una forma corta de poner la máscara de red, por ejemplo `/24` es lo mismo que `255.255.255.0`
Puerta de enlace	El equipo por donde sale el tráfico hacia internet, que en casa es el router
DNS	El servicio que traduce los nombres (`google.com`) a direcciones IP
DNS upstream	El DNS al que Pi-hole le pasa las consultas que no bloquea, que al principio era Cloudflare y ahora es Unbound, en el mismo contenedor
Lista de bloqueo	Una lista de dominios de anuncios y rastreadores que Pi-hole bloquea

HaGeZi	Un proyecto que hace listas de bloqueo DNS en varios niveles (de Light a Ultimate) y otras listas más concretas como TIF o DoH
Gravity	Lo que usa Pi-hole para bajar todas las listas de bloqueo y juntarlas en su base de datos
Unbound	Un DNS que busca las direcciones él solo, sin depender de un DNS de fuera como Cloudflare
Resolvedor recursivo	Un DNS que encuentra un nombre haciendo todo el camino, primero los servidores raíz, luego el dominio de primer nivel y al final el servidor del dominio
Servidores raíz	Los servidores que están arriba del todo en el sistema DNS, y saben a quién hay que preguntar por cada dominio de primer nivel (`.com`, `.net`, `.es`…)
DNSSEC	Unas firmas en las respuestas DNS que sirven para comprobar que nadie las ha cambiado por el camino
DNS cifrado (DoH)	DNS que va cifrado por HTTPS, y algunos navegadores y apps lo usan por su cuenta y así se saltan Pi-hole
DHCP	El servicio del router que reparte las IPs solo, y una *reserva* hace que un equipo reciba siempre la misma IP
Dirección MAC	El identificador único de una tarjeta de red, que el router usa para saber a qué equipo le da cada IP reservada
Puente de red (vmbr0)	Un switch virtual que conecta los contenedores y las VMs con la red de casa
Repositorio	El servidor de donde el sistema baja los programas y las actualizaciones
Docker	Un programa que ejecuta aplicaciones empaquetadas en *contenedores Docker*, con todo lo que necesitan para funcionar
Imagen Docker	Un paquete ya listo con una aplicación y todo lo que necesita, por ejemplo `vaultwarden/server`
Docker Compose	Una herramienta para poner uno o varios contenedores Docker en un archivo (`compose.yaml`) y arrancarlos con un solo comando
Tailscale	Una VPN que funciona con WireGuard y conecta tus dispositivos entre ellos sin abrir puertos en el router
Tailnet	La red privada que forman todos los dispositivos que están en la misma cuenta de Tailscale
MagicDNS	Una función de Tailscale que le pone nombre a cada máquina de la tailnet (por ejemplo `vaultwarden.tailxxxxxx.ts.net`)
Clave de host (SSH)	La clave que identifica a un servidor SSH, y su *huella* (fingerprint) sirve para comprobar que te estás conectando al servidor que toca
Contraseña maestra	La contraseña con la que se cifra la caja fuerte de Vaultwarden, y si se olvida no se puede recuperar
Subnet router	Una máquina de la tailnet que deja entrar a una red entera (por ejemplo la de casa) a los demás dispositivos de Tailscale
Reenvío IP	Cuando un Linux puede pasar paquetes de una red a otra como si fuera un router (en inglés *IP forwarding*)
Ruta más concreta	Cuando hay varios caminos para llegar al mismo sitio gana el que tiene el número más alto después de la barra, por ejemplo una `/24` gana a una `/23`

Node.js	Lo que se usa para ejecutar programas hechos en JavaScript fuera del navegador, y es con lo que funciona Uptime Kuma
npm	El gestor de paquetes de Node.js, que baja e instala las librerías que necesita un programa
systemd (servicio)	Lo que arranca y vigila los programas en Linux, y un *servicio* es un programa que systemd arranca solo y lo vuelve a encender si falla
Monitor	En Uptime Kuma, cada comprobación que se repite cada cierto tiempo de un servicio (ping, web, DNS…)
SMTP	El protocolo para mandar correos, y Uptime Kuma lo usa para mandar los avisos con Gmail
Contraseña de aplicación	Una contraseña que crea Google para que un programa use la cuenta sin saber la contraseña de verdad, y se puede borrar cuando quieras
arping	Una herramienta que pregunta en la red quién tiene una IP, y sirve para ver si una IP está libre
Transcodificación	Convertir un vídeo a otro formato o calidad mientras se está viendo, para que lo pueda reproducir un dispositivo que no soporta el original
Reproducción directa	Cuando el dispositivo reproduce el archivo tal cual sin convertir nada, que es lo que menos carga al servidor
HDR / tone mapping	El HDR es un formato de vídeo con más brillo y más color, y el *tone mapping* lo pasa a imagen normal (SDR) para las pantallas que no lo soportan
VA-API	Lo que usa Linux para que la gráfica convierta vídeo, y QuickSync funciona encima de esto
Punto de montaje	Un espacio de almacenamiento extra que se le añade a un contenedor en una carpeta concreta, como `/srv/media`
SFTP	Pasar archivos por SSH de forma cifrada, que es lo que usa WinSCP para subir las películas
Quick Sync	Una función de las gráficas Intel para convertir vídeo con la gráfica y no cargar el procesador